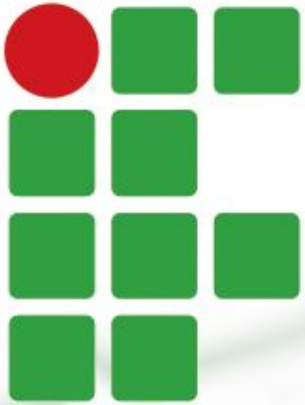


INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Sistemas Distribuídos

- Concorrência -

Sistemas Distribuídos

…mas antes…

UMA BREVE VIAGEM NO TEMPO

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Sistemas Distribuídos

Apollo Guidance Computer (AGC) / Apollo 11



Processador:

- Feito com lógica TTL (Transistor-Transistor Logic)
- Frequência de clock: **1,024 MHz** (sim, apenas 1 megahertz!)
- CPU de 16 bits, com algumas operações em 15 bits

Memória RAM (volátil):

- **2 KB (2048 palavras)** de RAM chamada **"erasable memory"**

Memória ROM (não volátil):

- **32 KB (36.864 palavras)** de memória chamada **"core rope memory"** (programada fisicamente com fios entrelaçados)

Capacidade de instruções:

- Suportava cerca de **34 instruções** (instruções básicas como ADD, SUB, etc.)

Sistema Operacional:

- Um sistema cooperativo em tempo real, com multitarefa baseada em prioridades

Unidade de Entrada/Saída:

- Usava um dispositivo chamado **DSKY (Display and Keyboard)** — a principal interface com os astronautas

Em um passado **não tão distante...**

HEWLETT PACKARD

16 MB Memory

Microsoft 28.8 Fax/Modem

Windows 95

Includes All This Great Software and More!

$2199.99

Monitor Sold Separately

120MHz PENTIUM® PROCESSOR MULTIMEDIA SYSTEM

- 1.6 gigabyte hard drive
- 16 MB memory
- 1 MB video memory
- Quad-speed CD-ROM drive
- 16-bit stereo sound card and speakers
- 28.8 internal fax/modem
- Telephone answering system
- Full-duplex speakerphone
- MPEG, full-screen full-motion video
- Pre-loaded and CD-ROM software (7055)

14" .28 Dot Pitch Super VGA Non-Interlaced Color Monitor with Audio Amplifier and Bass Boost (12.52" viewable image size/3858) $349.99

COMPAQ

75MHz Pentium® Processor

$1699.97 COMPLETE SYSTEM

Presario 5528 Integrated Multimedia Computer System

Limited Quantities. No Rainchecks.

- 8MB RAM
- 1060MB hard drive
- Quad speed CD-ROM drive
- 19,200 bps modem/ 14,400 bps fax
- 1MB video DRAM
- 14" (11.9" viewable) .28dp monitor

Other Features

- MPEG: Full Motion Video
- Spatializer® 3-D Surround Sound
- Phone answering system
- Full duplex speakerphone

214150009

Pre-Installed Software!

PLUS 34 MORE TITLES!

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Em um passado
(não tão distante)...




Pentium® III Xeon™ Processor
Powering Servers and Workstations to New Levels of Performance
intel inside
pentium III xeon
667 MHz /256KB CACHE
intel
256KB Cache
Attached Heatsink
This product is intended to be professionally installed.
pentium III xeon
intel
Supports dual processing



Vostro Small Desktop

4.3 (1370) Fazer uma pergunta

- 10ª geração de Intel® Core™ i7-10700 (8-core, cache de 16MB, 2.9GHz até 4.8GHz)
- Windows 11 Home Single Language, Português
  (A Dell Technologies recomenda o Windows 11 Pro para empresas | Upgrade gratuito para o Windows 11*)
- Placa gráfica UHD Intel® Graphics
- Memória de 8GB (1x8GB), DDR4, 2933MHz, Expansível até 64GB (2 slots UDIMM, 1 slot livre)
- SSD de 512GB PCIe NVMe M.2
- Portas e slots

Preço **R$ 6.098,00**

Ofertas especiais



MEMÓRIA INTERNA
256GB

CHIP
DUAL CHIP
NANO-SIM

MEMÓRIA RAM
12GB

RECURSOS
LEITOR BIOMÉTRICO

CONECTIVIDADE
BLUETOOTH, WI-FI, NFC

CAPACIDADE BATERIA
5000mAh

PROCESSADOR
OCTA-CORE

COR
PRETO

TELA
6,8" QHD+

RESOLUÇÃO CÂMERAS
TRASEIRA: QUÁDRUPLA
FRONTAL: 40MP

Itautec
Abra o olho para o futuro.





Nome do produto	Data de lançamento	Número de núcleos	Frequência turbo max	Frequência baseada em processador	Cache	TDP
Processador Intel® Xeon® Platinum 8490H (cache de 112,5 M, 1,90 GHz)	Q1'23	60	3.50 GHz	1.90 GHz	112.5 MB	350 W
Processador Intel® Xeon® Platinum 8480+ (cache de 105 M, 2,00 GHz)	Q1'23	56	3.80 GHz	2.00 GHz	105 MB	350 W
Processador Intel® Xeon® Platinum 9282 (cache de 77 M, 2,60 GHz)	Q2'19	56	3.80 GHz	2.60 GHz	77 MB Intel® Smart Cache	400 W
Processador Intel® Xeon® Platinum 8470N (cache de 97,5 M, 1,70 GHz)	Q1'23	52	3.60 GHz	1.70 GHz	97.5 MB	300 W
Processador Intel® Xeon® Platinum 8471N (cache de 97,5 M, 1,80 GHz)	Q1'23	52	3.60 GHz	1.80 GHz	97.5 MB	300 W
Processador Intel® Xeon® Platinum 8470Q (cache de 105 M, 2,10 GHz)	Q1'23	52	3.80 GHz	2.10 GHz	105 MB	350 W
Processador Intel® Xeon® Platinum 8470 (cache de 105 M, 2,00 GHz)	Q1'23	52	3.80 GHz	2.00 GHz	105 MB	350 W



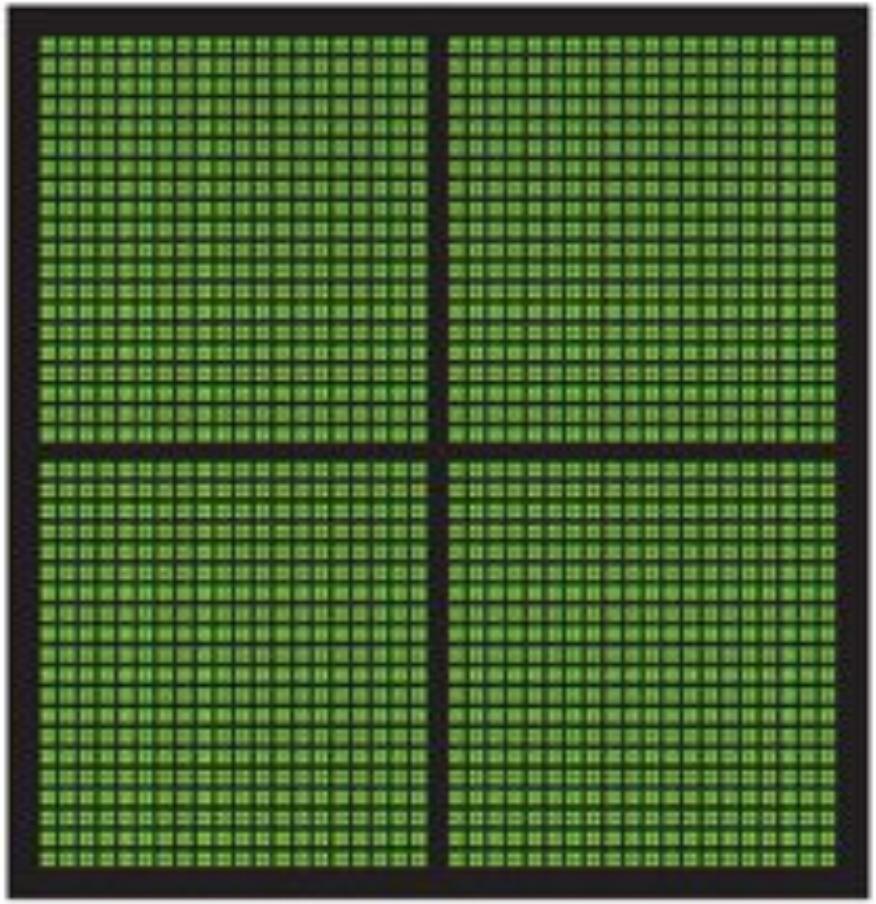
1970 1980	Single Core	Processamento sequencial, ideal para tarefas simples.
2005	Dual Core	Dois threads simultâneos, melhora multitarefa básica.
2006	Quad Core	Quatro núcleos, eficiente para mais recursos.
2010	Hexa Core	Seis núcleos, ótimo para VMs e compilação.
2012	Octa Core	Oito núcleos, ideal para cargas pesadas.
2015	Deca Core	Dez núcleos, perfeito para virtualização.
2017	Dodeca Core	Doze núcleos, para servidores exigentes.
2019	Hexadeca Core	Dezesseis núcleos, para simulações complexas.
2021	24 núcleos	Para big data e IA.
2023	32 núcleos	Para nuvem e alta densidade.



1970 1980	Single Core	Processamento sequencial, ideal para tarefas simples.







CPU
MULTIPLE CORES

GPU
THOUSANDS OF CORES

1970 1980	Single Core	Processamento sequencial, ideal para tarefas simples.



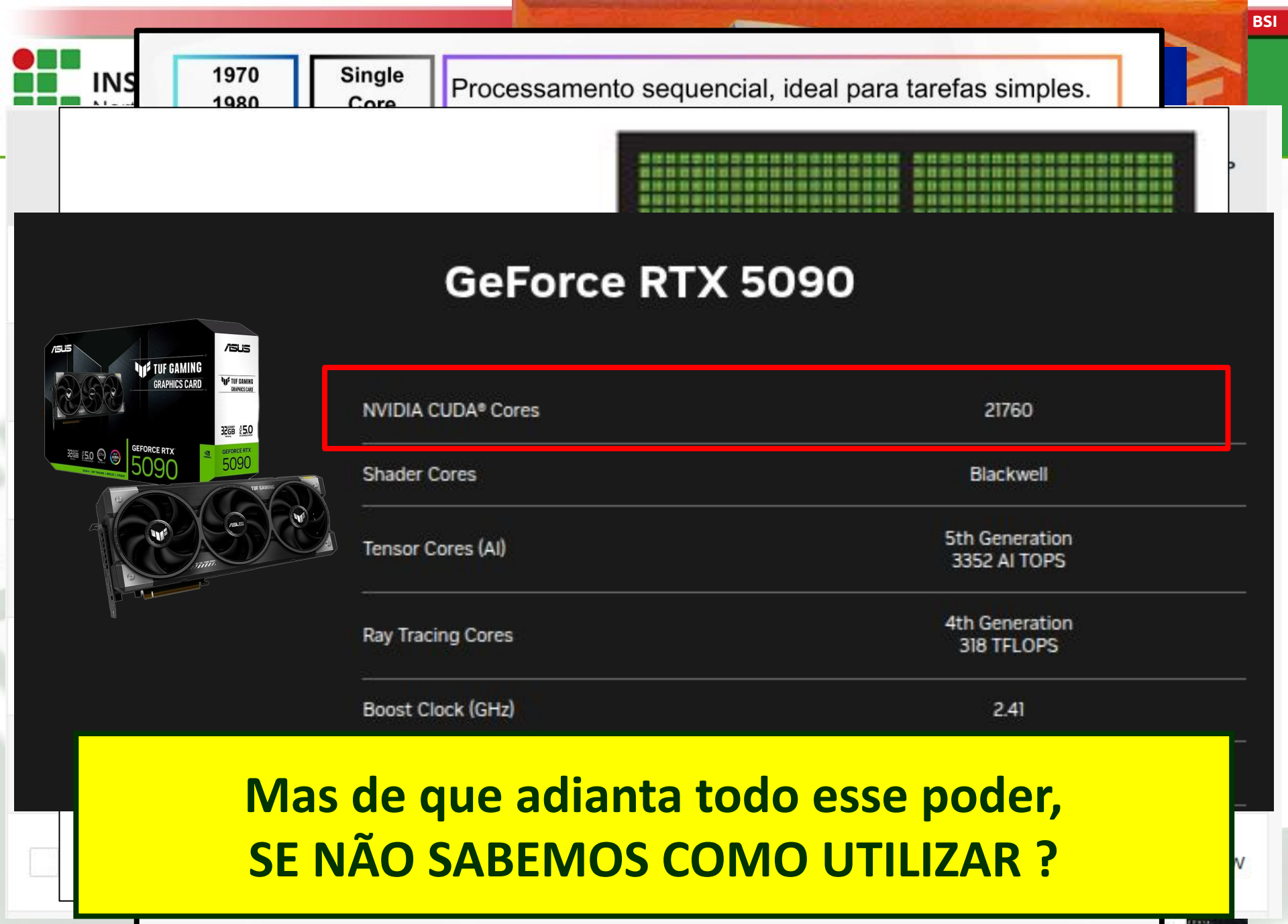
GeForce RTX 5090



NVIDIA CUDA® Cores	21760
Shader Cores	Blackwell
Tensor Cores (AI)	5th Generation 3352 AI TOPS
Ray Tracing Cores	4th Generation 318 TFLOPS
Boost Clock (GHz)	2.41
Base Clock (GHz)	2.01

MULTIPLE CORES THOUSANDS OF CORES

1970 1980	Single Core	Processamento sequencial, ideal para tarefas simples.



Mas de que adianta todo esse poder,
SE NÃO SABEMOS COMO UTILIZAR ?

1970 | Single | Processamento sequencial, ideal para tarefas simples.

Terminal

Arquivo Editar Ver Pesquisar Terminal Ajuda

```
Python 3.11.6
4444444444444423 é Primo
5000111000222021 é Primo
6666666666666719 é Primo
9999999999999917 é Primo
Tempo Processamento Síncrono = 28.48129677772522
5000111000222021 é Primo
4444444444444423 é Primo
6666666666666719 é Primo
9999999999999917 é Primo
Tempo Processamento Paralelo = 17.293292999267578
```

Lab 03.1

TDP

350 W

350 W

400 W

300 W

300 W

Mas de que adianta todo esse poder,
SE NÃO SABEMOS COMO UTILIZAR ?

1970 | Single | Processamento sequencial, ideal para tarefas simples.

Terminal

Arquivo Editar Ver Pesquisar Terminal Ajuda

```
Python 3.11.6
4444444444444423 é Primo
5000111000222021 é Primo
6666666666666719 é Primo
9999999999999917 é Primo
Tempo Processamento Síncrono = 28.48129677772522
5000111000222021 é Primo
4444444444444423 é Primo
6666666666666719 é Primo
9999999999999917 é Primo
Tempo Processamento Paralelo = 17.293292999267578
```

Lab 03.1

TDP

350 W

350 W

400 W

300 W

300 W

Mas de que adianta todo esse poder,
SE NÃO SABEMOS COMO ~~UTILIZAR~~ PROGRAMAR ?

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

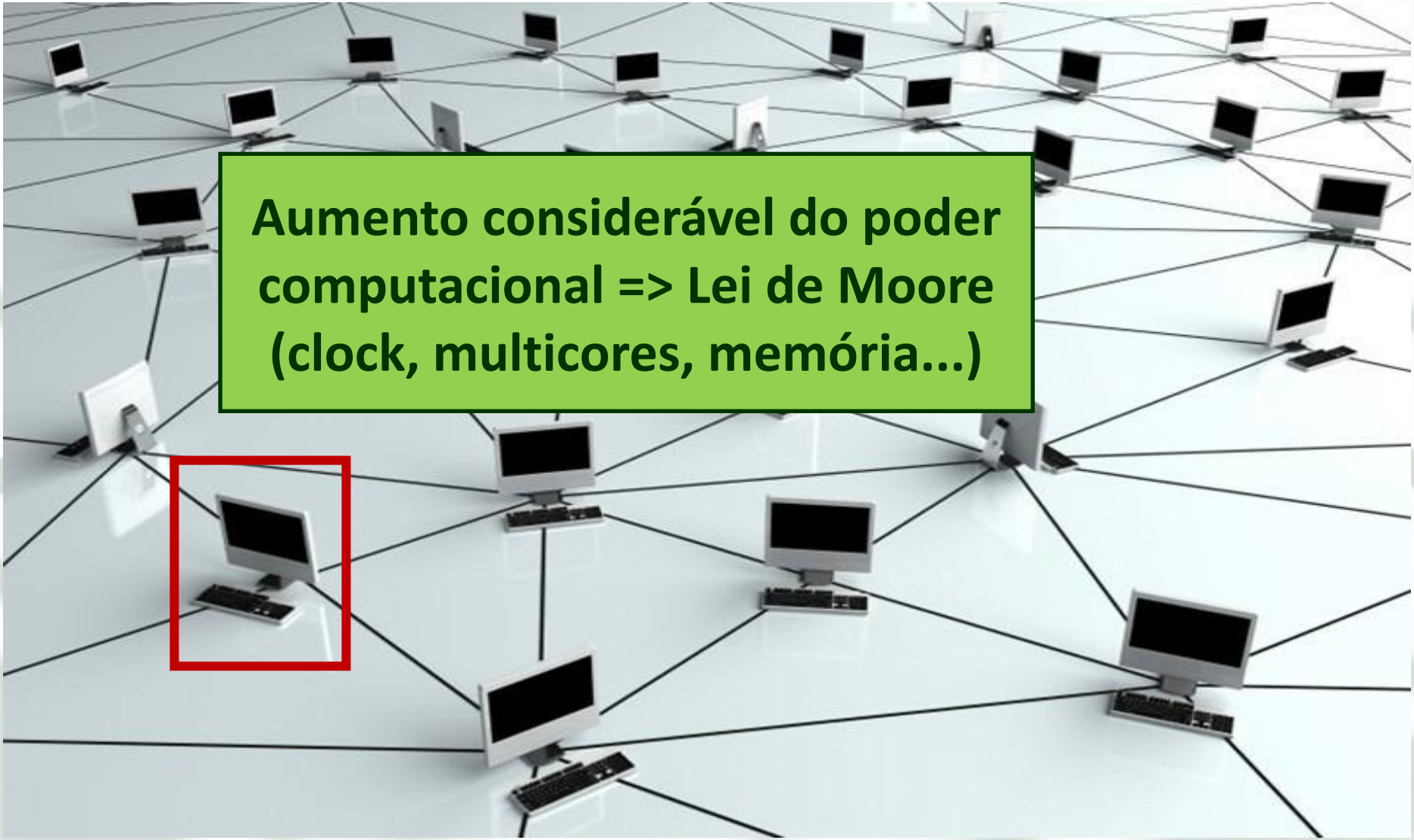
Vídeo Complementar

Assista ao vídeo complementar abaixo.



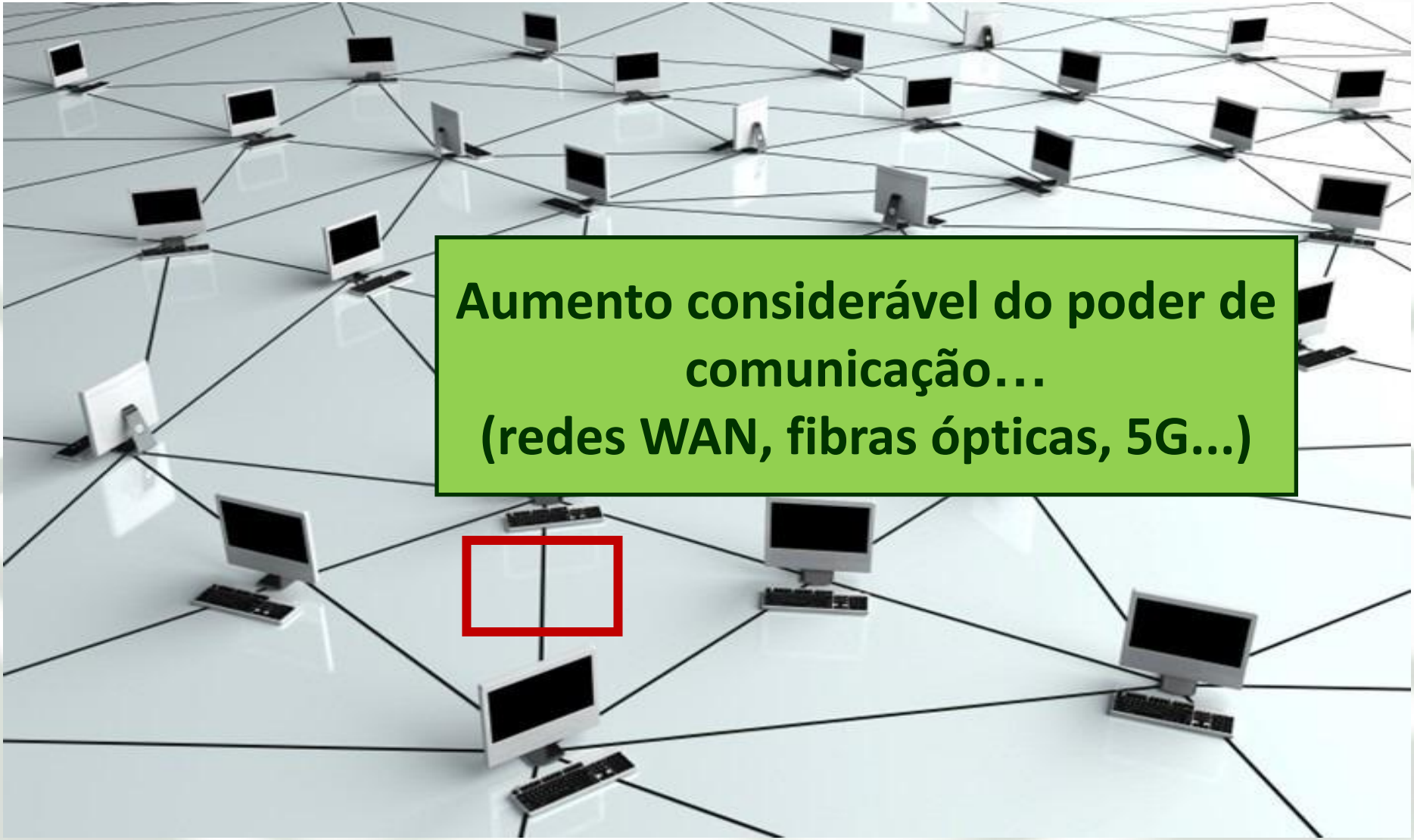
INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Sistemas Distribuídos



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

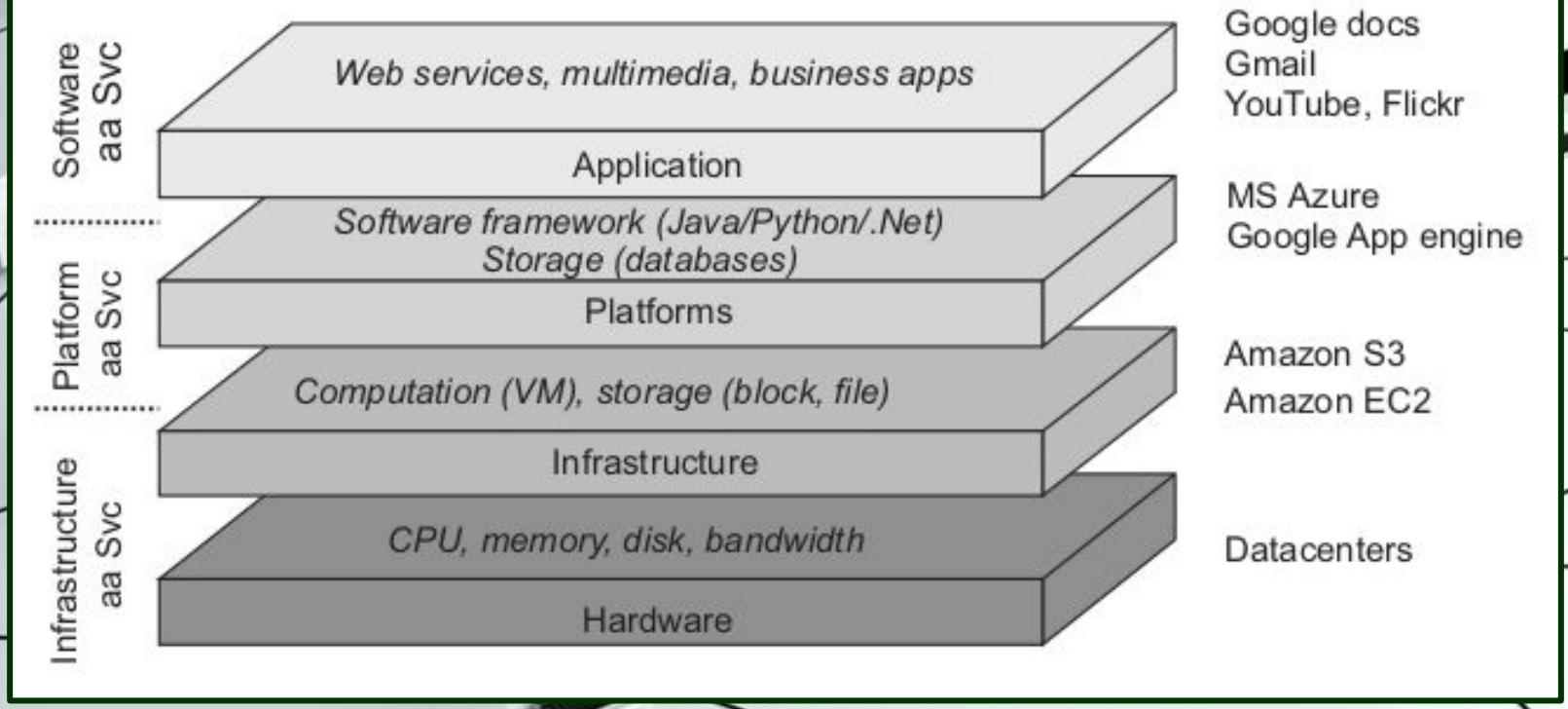
Sistemas Distribuídos



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Sistemas Distribuídos

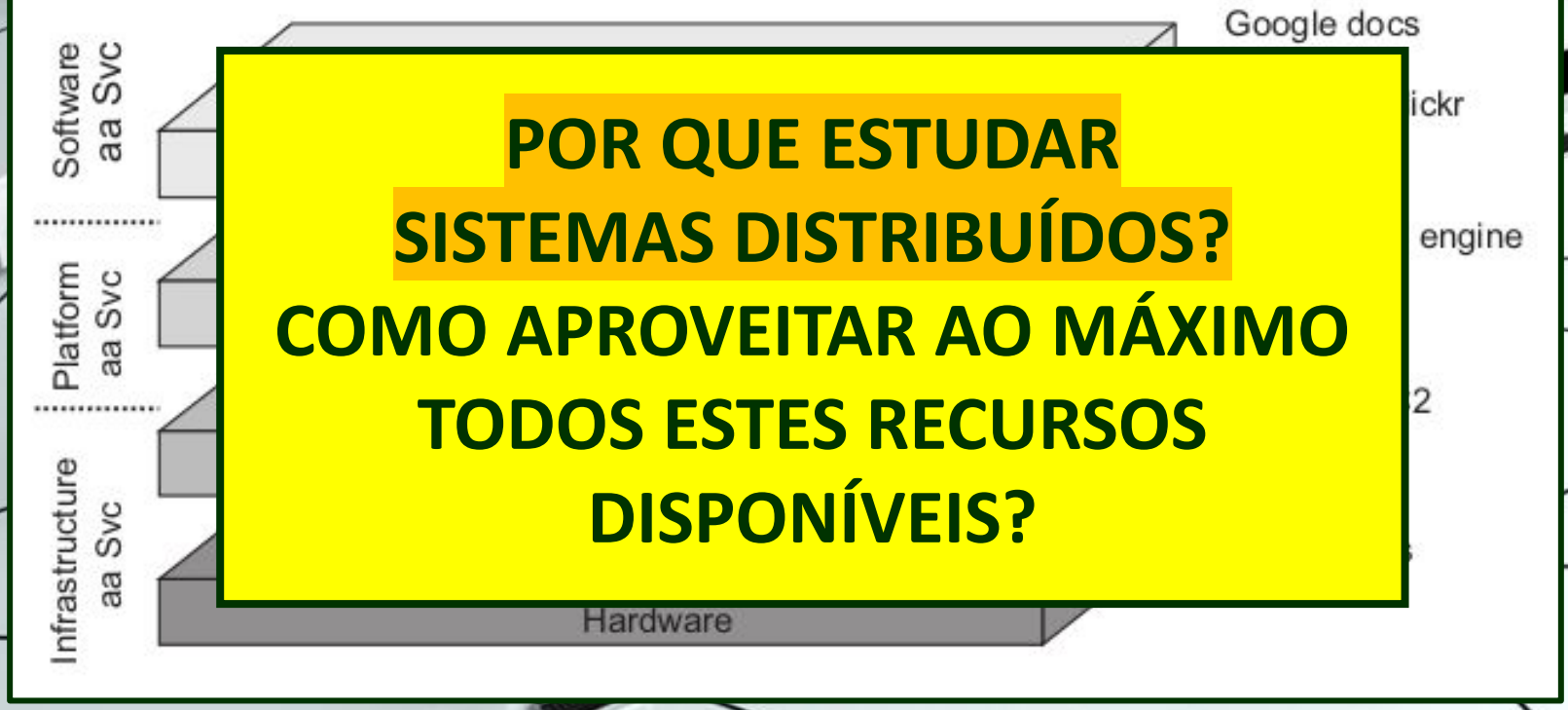
Cloud Computing



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Sistemas Distribuídos

Cloud Computing



POR QUE ESTUDAR SISTEMAS DISTRIBUÍDOS?
COMO APROVEITAR AO MÁXIMO TODOS ESTES RECURSOS DISPONÍVEIS?

Sistemas Distribuídos

Não seria possível conceber

SISTEMAS DISTRIBUÍDOS sem a

existência de **CONCORRÊNCIA**

Sistemas Operacionais

- Evolução dos **Sistemas Operacionais**
  - Monotarefas
  - Multitarefas
  - Multiprocessados

Sistemas Operacionais

- Os sistemas operacionais modernos são construídos sobre três pilares fundamentais:

Virtualization (Process and Memory)	Concurrency	Persistance
1.OS gives every process a feeling that it has own CPU	1.Multiple process can run at the same time without resulting in problems	1.Disk is an I/O device. It needs to be managed and handled.
2.Every process feels that its enough memory	2. OS Provides mechanisms to make them work together	2.OS handles interactions with the disk and performs storage management

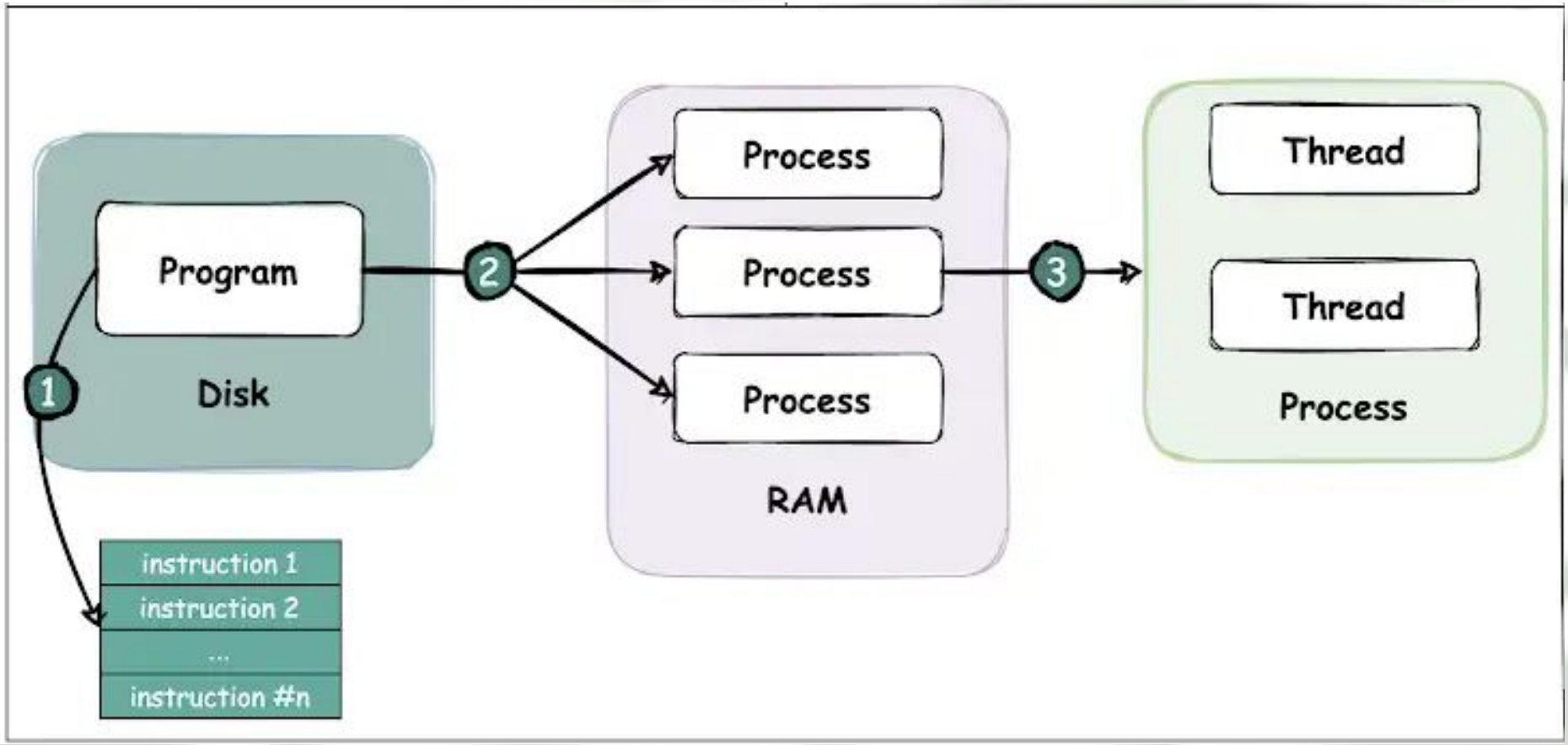
CS3.301 Sistemas Operacionais e Redes, IIIT Hyderabad

Processo

- Processos são, resumidamente, **programas em execução**! (*outros termos semelhantes: job, task*)
- Estudo do ponto de vista de Sistemas Operacionais:
  - Gerenciamento dos Processos (Início, Meio e Fim)
  - Escalonamento de Processos (Trocas de Contexto)
- Estudo do ponto de vista dos **Sistemas Distribuídos**:
  - Como aproveitar ao máximo os recursos computacionais disponíveis para atender os processos?

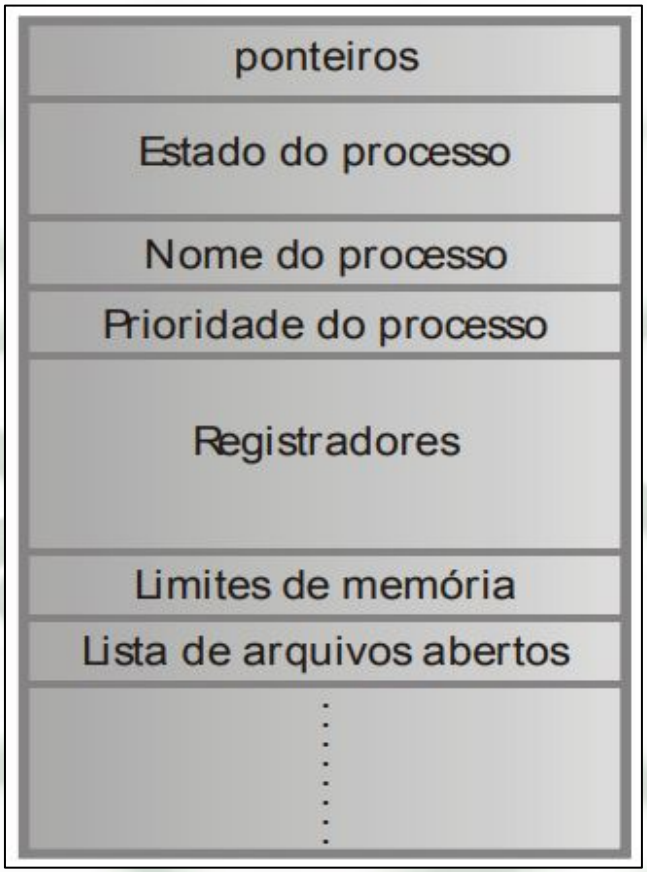
Processos vs. Threads

- Programas vs. Processos vs. Threads



PCB

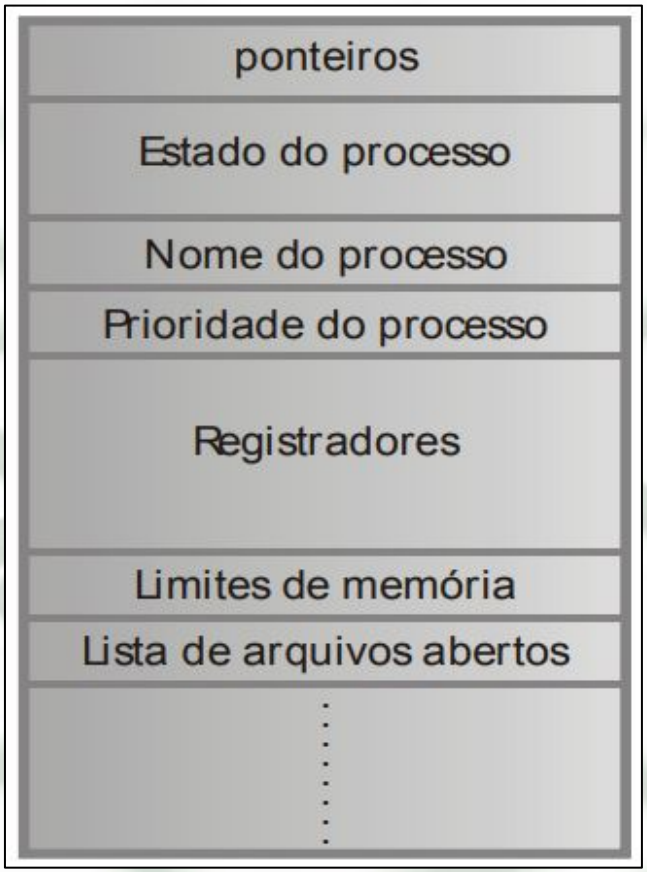
- ***Process Control Block***
  - Estrutura de Dados mantida pelo S.O para cada processo executado.
  - Armazena todas as informações necessárias para a execução.
  - Totalmente independentes entre si.
  - Grande quantidade de dados atualizada a cada instante.
- Os PCB residem na memória principal do dispositivo => ***Kernel Space*** (Memória dedicada ao S.O.)



PCB

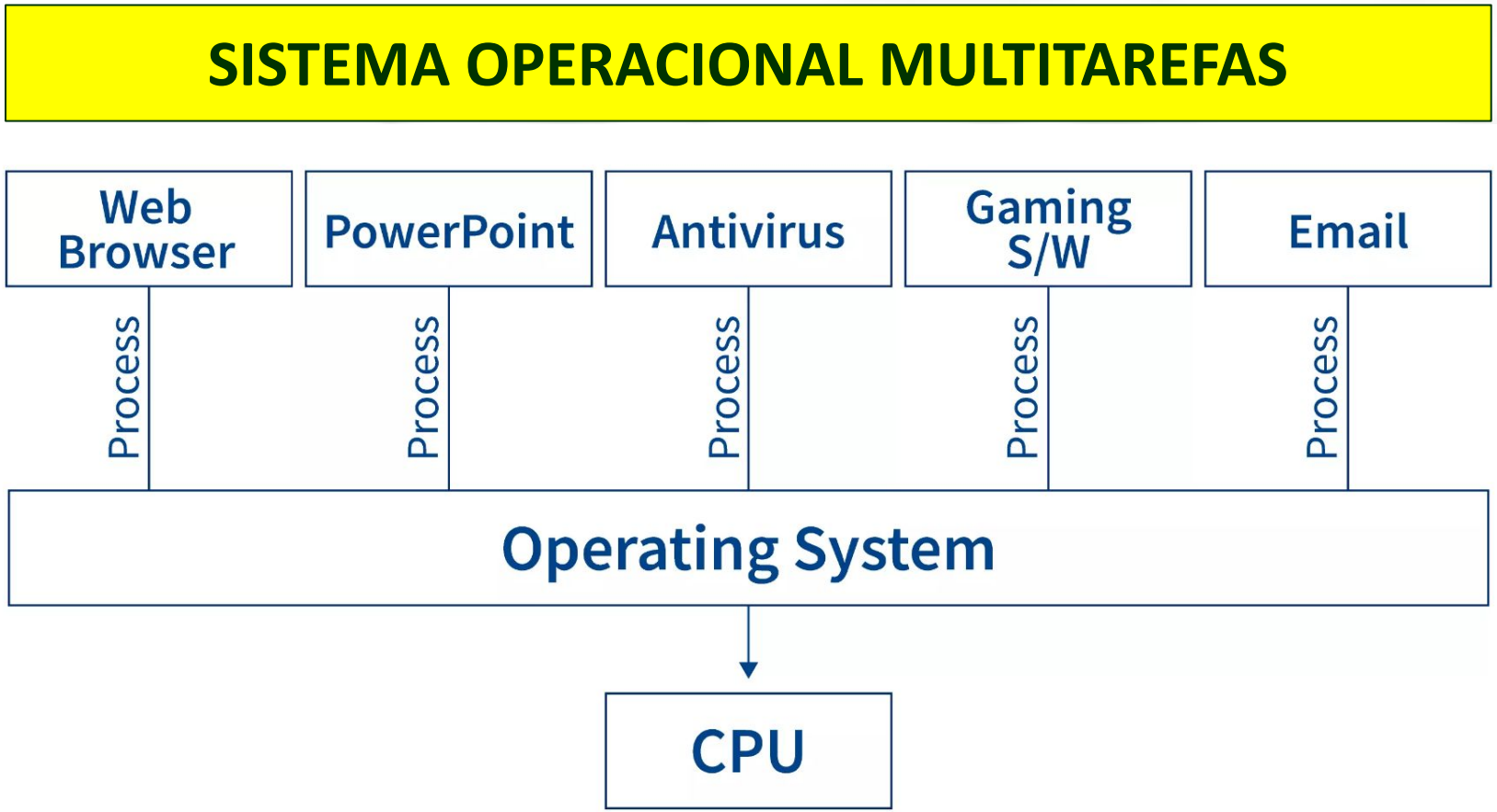
- **Ao sistema operacional cabe a tarefa de gerenciar os PCBs...**
- Os processos devem compartilhar os *hardwares* entre si (memória, CPU, disco, I/O, rede, etc...)
- Este compartilhamento é feito **pela alternância dos PCBs em execução num dado instante de tempo**:

ESCALONAMENTO



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Multitarefas

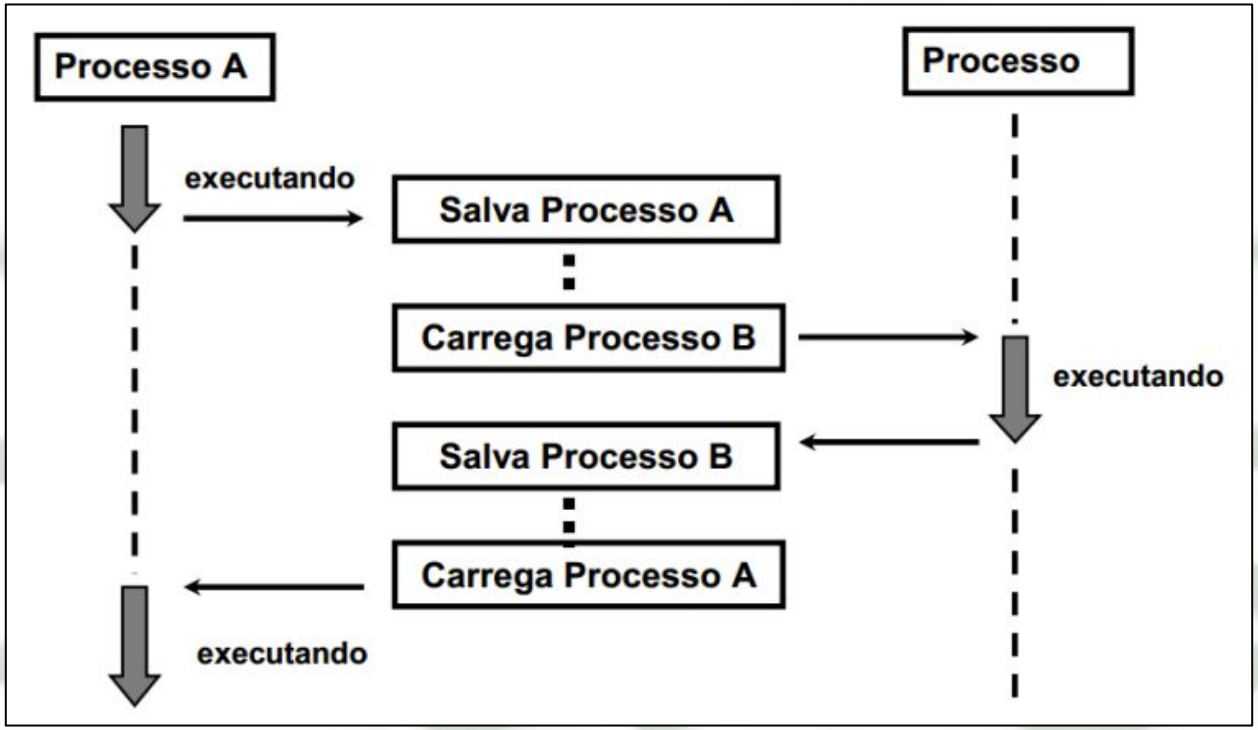


Troca de Contexto

- **Troca de Contexto** é o procedimento de alternar os processos em execução na CPU.
- Cada CPU (core) executa apenas um processo por vez.
- Isso gera uma sobrecarga de operações custosas (*overhead*) para o sistema (é o preço pago para ter um sistema multitarefas).
- S.O. deve colocar e retirar processos em execução:
  - **Ao retirar**, deve guardar todo o estado atual do PCB na memória RAM => *kernel space*.
  - **Ao colocar**, deve restaurar todo o PCB da memória RAM para execução na CPU.

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Troca de Contexto



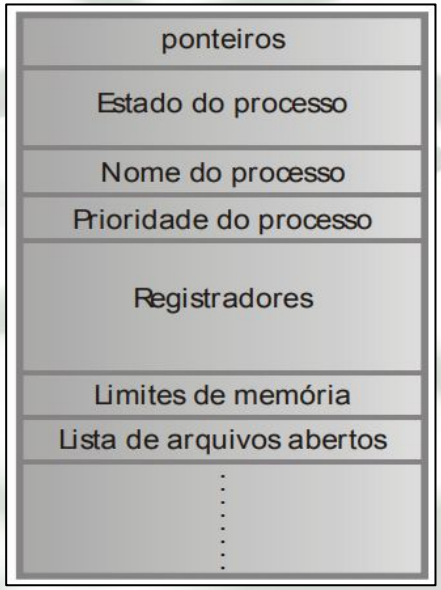
Sistemas Atuais: de 100 a 1000 trocas / segundo.

REALMENTE INCRÍVEL!!!

Informações do PCB

- Estado
  - Ativo/Executando => Na CPU
  - Pronto/Esperando => Aguardando sua vez
  - Em espera/Bloqueado => Aguardando I/O, Sync.
  - Inativo/Zumbi => Finalizado
- PID (Process ID)
- Usuário do Processo
- Prioridade de Execução
  - [-20, +20] #Conceito *Nice do Linux*

Etc...

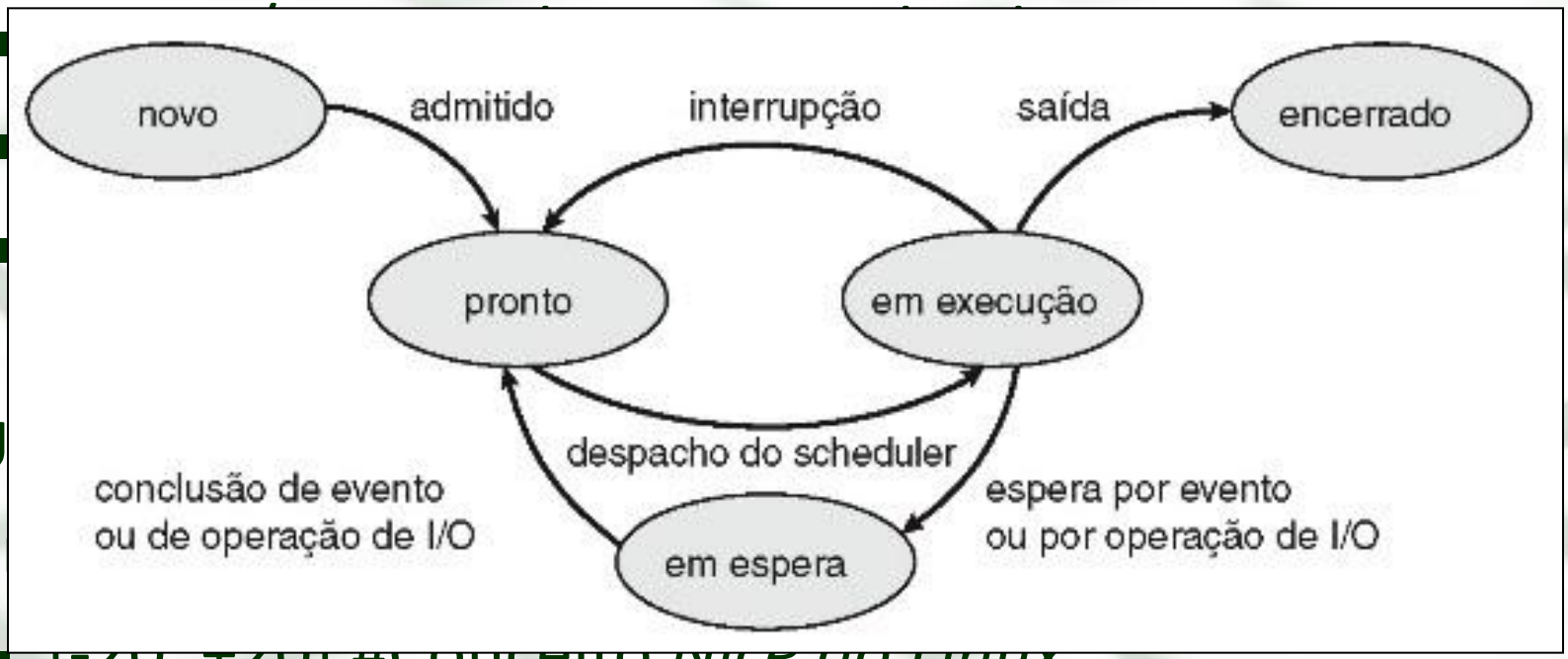


INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Estado do Processo

- Estado
  - Ativo/Executando => Na CPU
  -
  -
  -
- P
- U
- P
  - [-20, +20] #Conceito *Nice do Linux*

Etc...

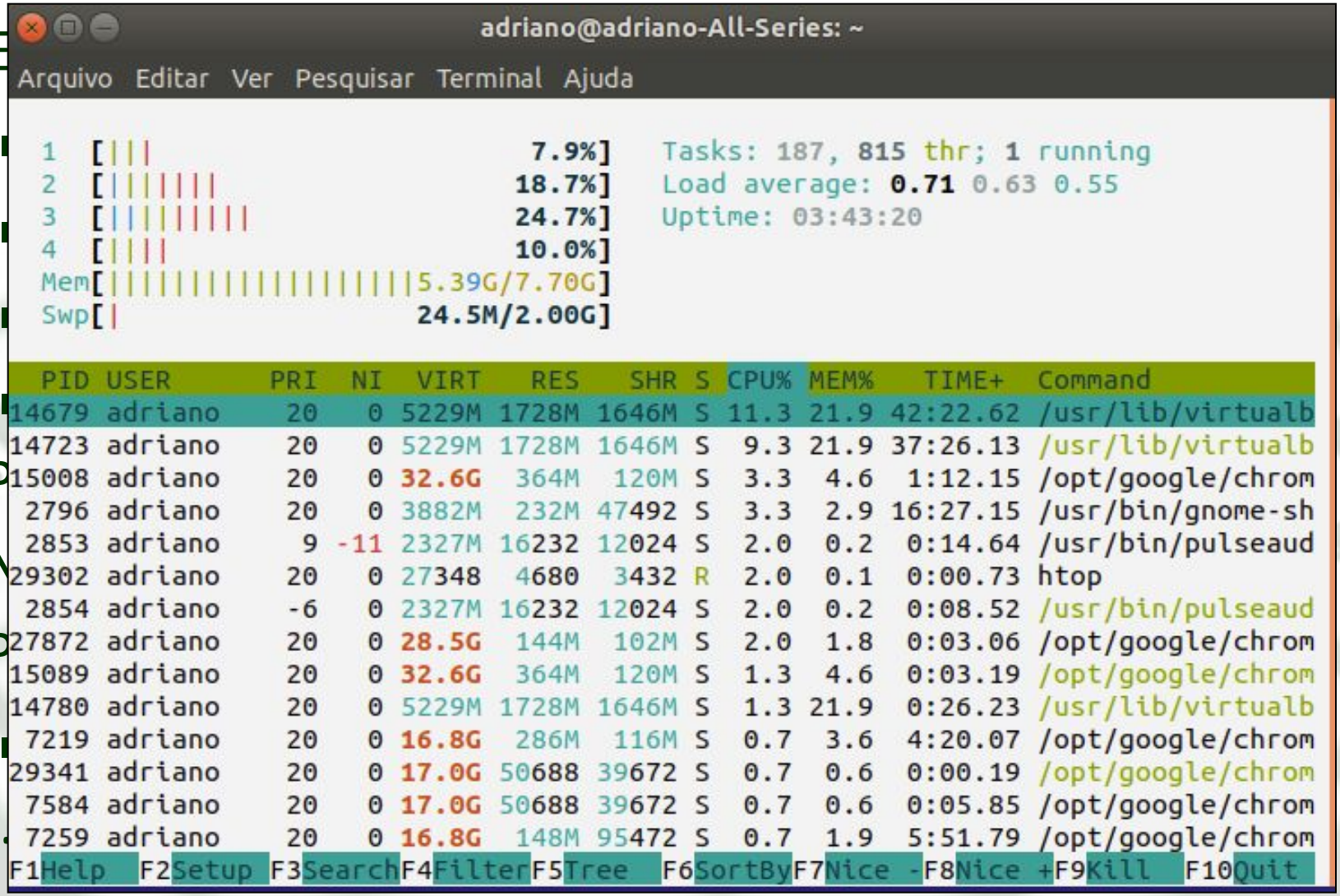


INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Gerenciador de Processos

- E
- P
- N
- P

Etc.



Outros Aspectos

- **Outros aspectos relevantes para Sistemas Distribuídos...**
- Em que estado um processo qualquer "passa" maior parte do tempo?
  - *CPU-Bound*
  - *IO-Bound*
- Quantos processos podem estar em estado Running simultaneamente?
- Qual unidade de escalonamento? Processos ou Threads?
- Processos podem se comunicar entre si?

Outros Aspectos

- **Outros aspectos relevantes para Sistemas Distribuídos...**
- Em que estado um processo qualquer "passa" maior parte do tempo?
  - *CPU-Bound*
  - *IO-Bound*

Veremos tudo isso em detalhes à frente...
MAS ANTES, E MAIS IMPORTANTE...

- Quantos processos podem estar em estado Running simultaneamente?
- Qual unidade de escalonamento? Processos ou Threads?
- Processos podem se comunicar entre si?

Concorrência e Paralelismo

Concorrência e **Paralelismo** são sinônimos?

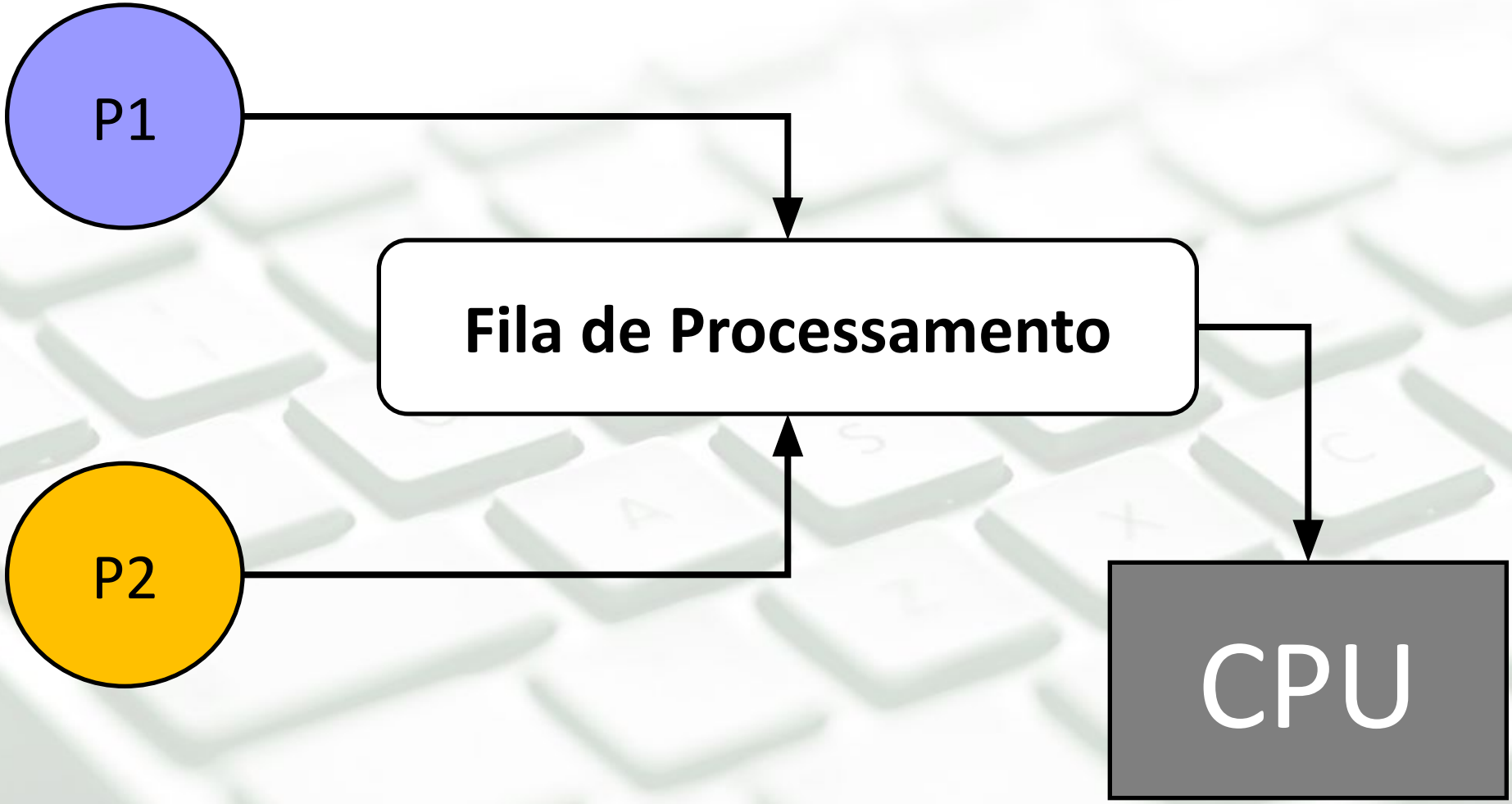
Concorrência

- **Computação multitarefa** é obtida a partir do suporte à **Concorrência**.
- **Concorrência** é o termo que utilizado quando processos **disputam o acesso a recursos compartilhados** (p.ex. CPU).

- Graças a este conceito, você tem a impressão que um computador "*faz várias coisas ao mesmo tempo*", o que não é necessariamente verdade.

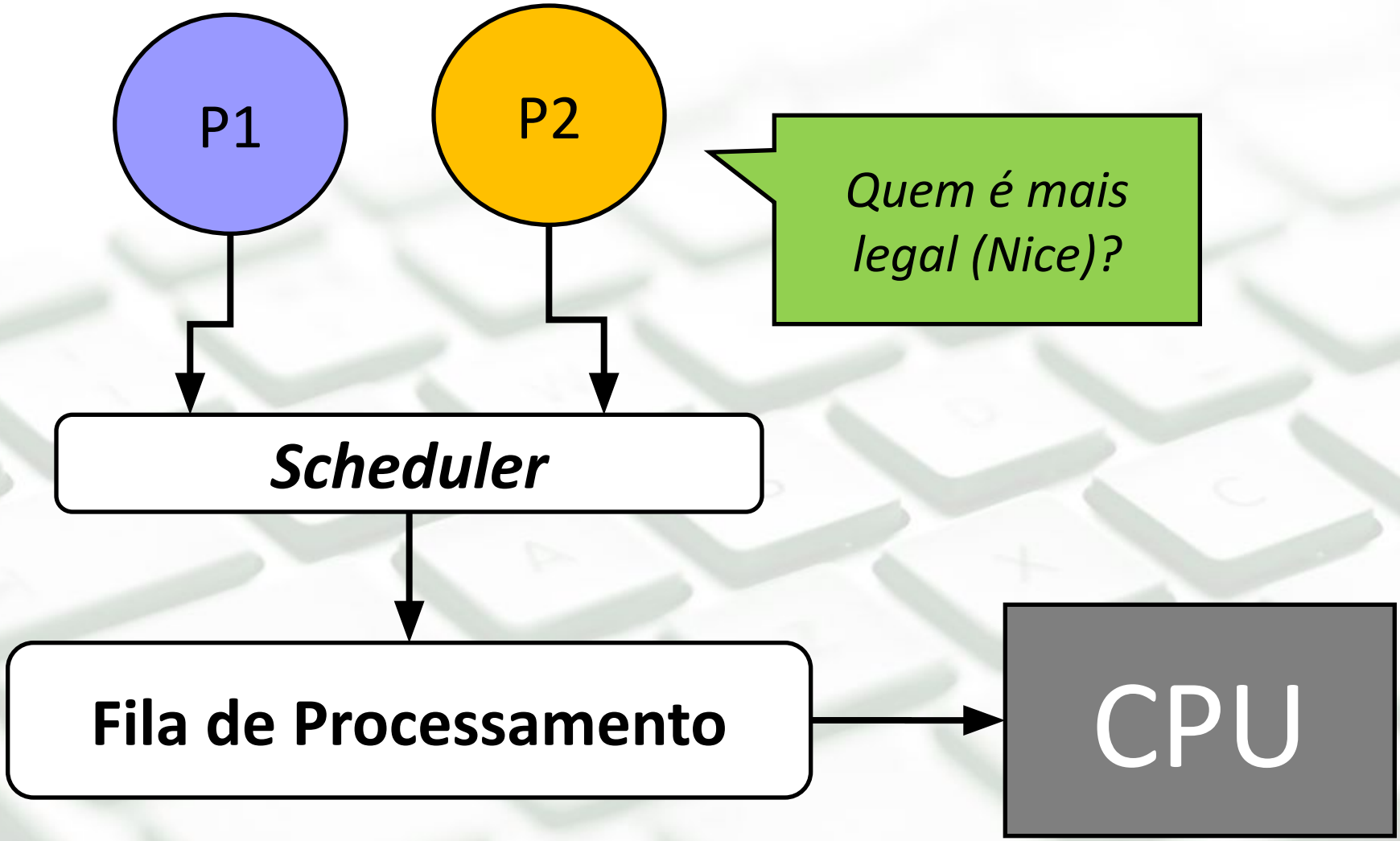
INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Concorrência



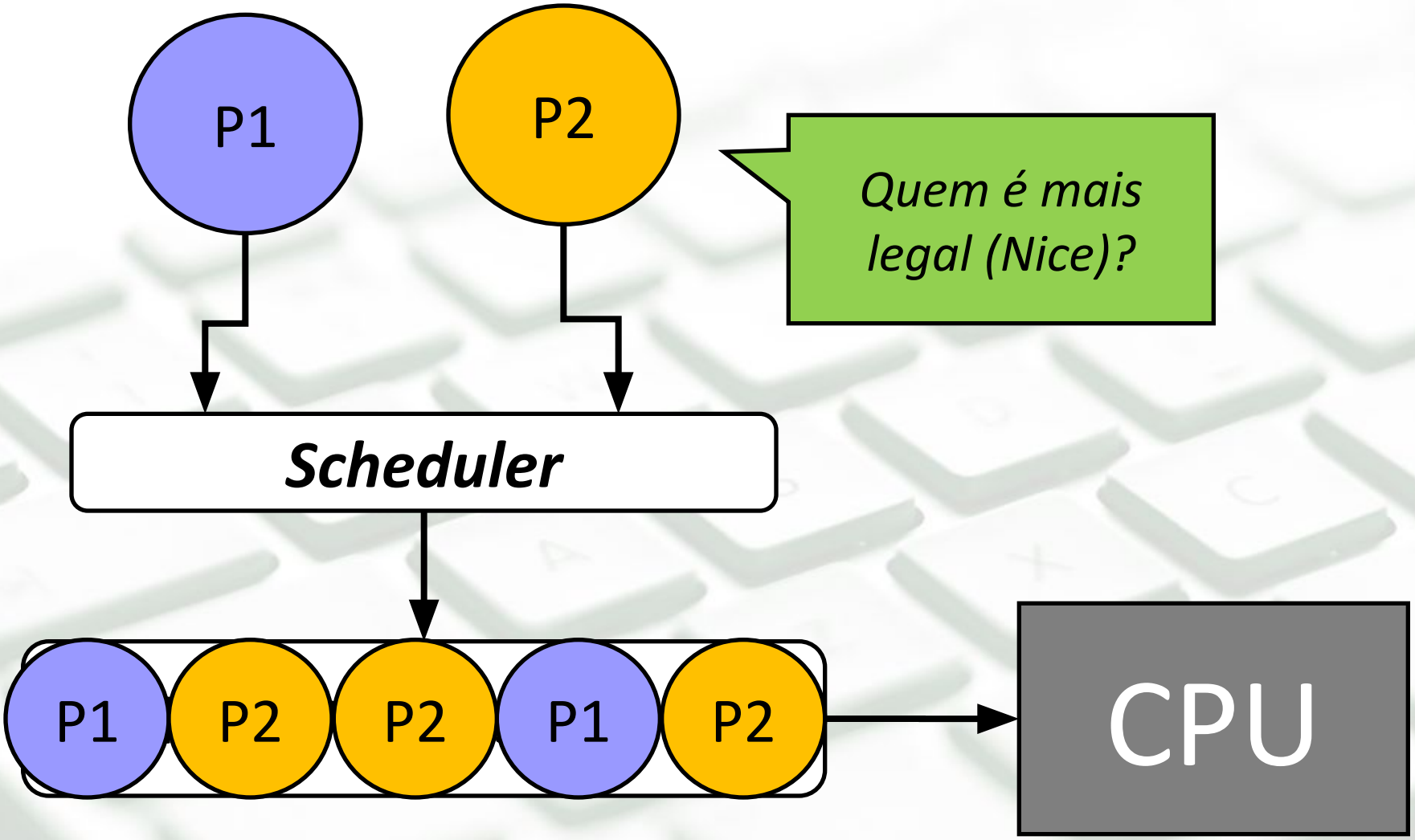
INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Escalonador



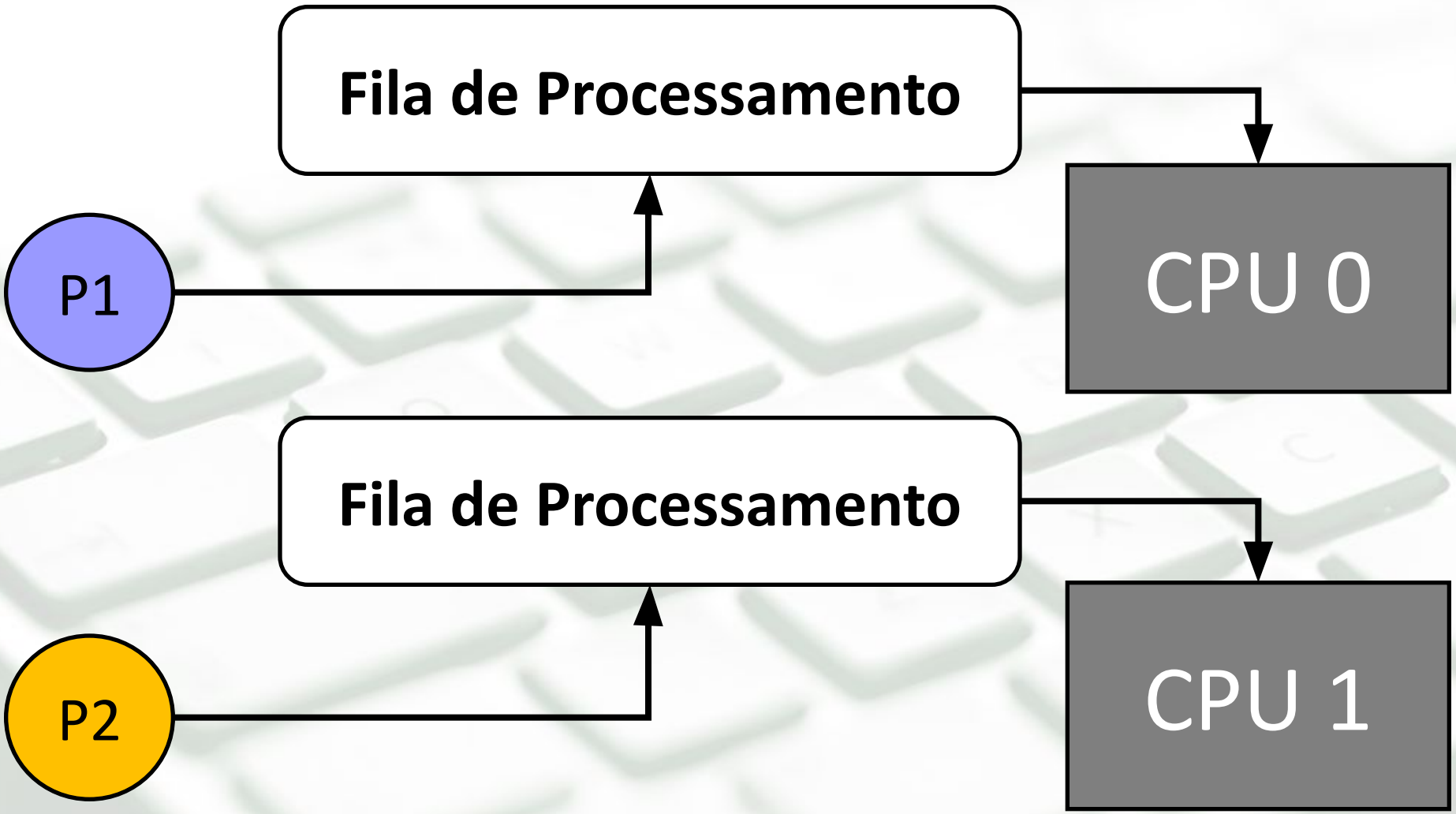
INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Escalonador



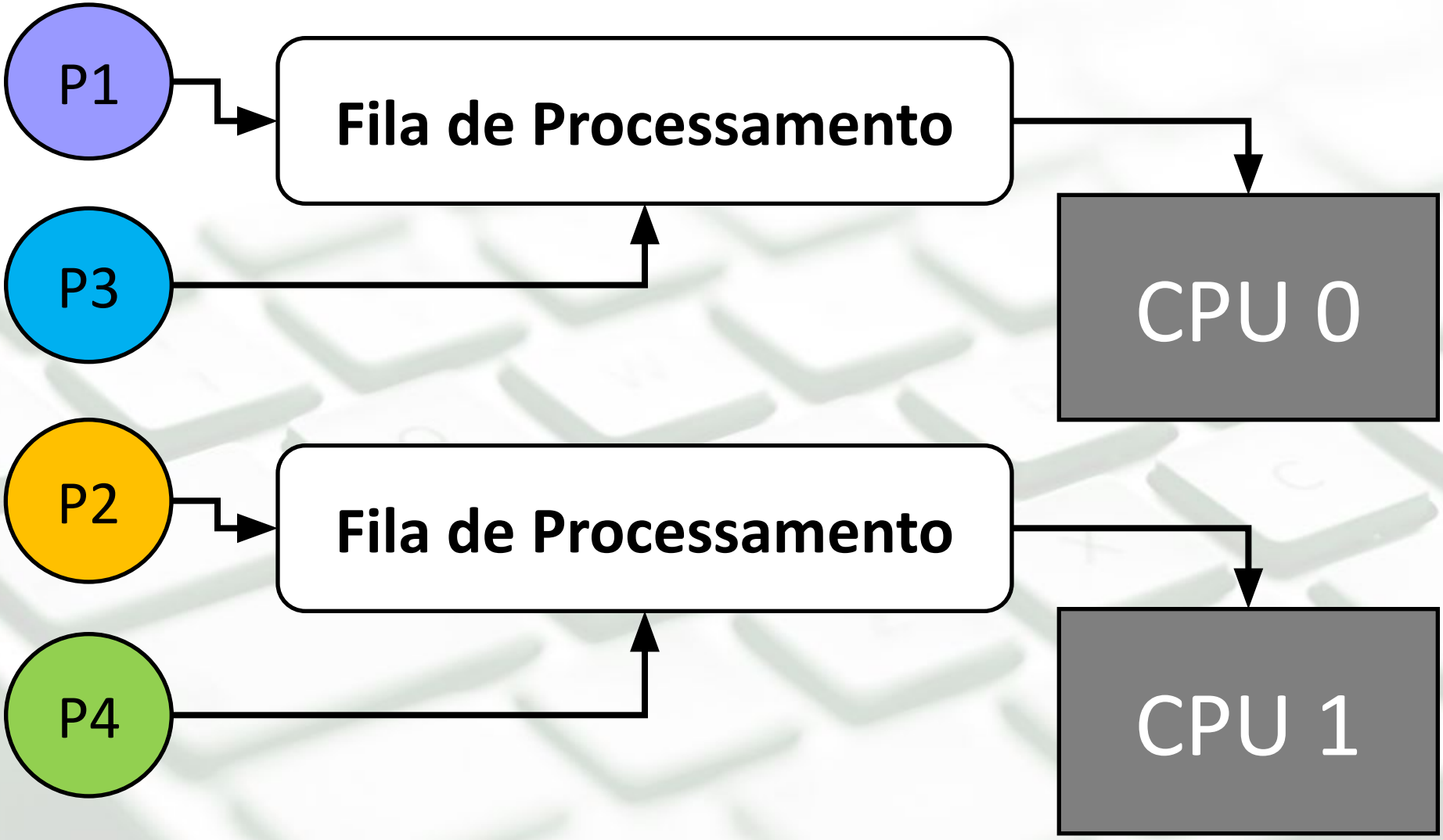
INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Paralelismo



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Paralelismo com Concorrência



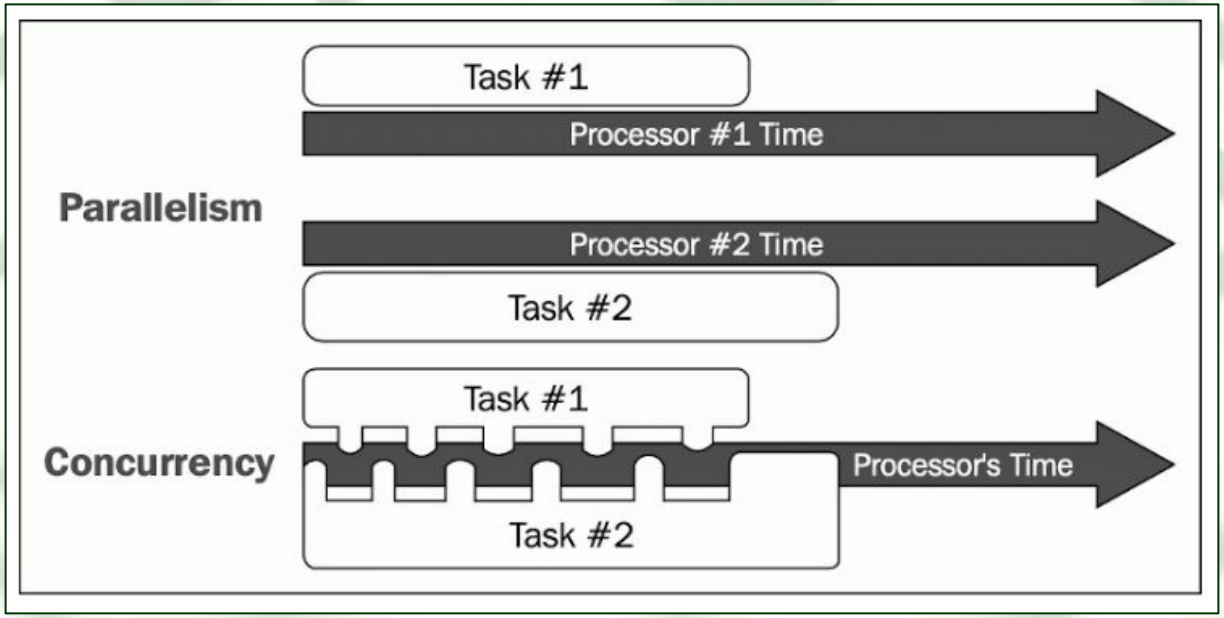
Filosofando...

- ***Concorrência*** *é sobre* ***lidar*** *com várias coisas ao mesmo tempo...*

- ***Paralelismo*** *é sobre* ***fazer*** *várias coisas ao mesmo tempo...*

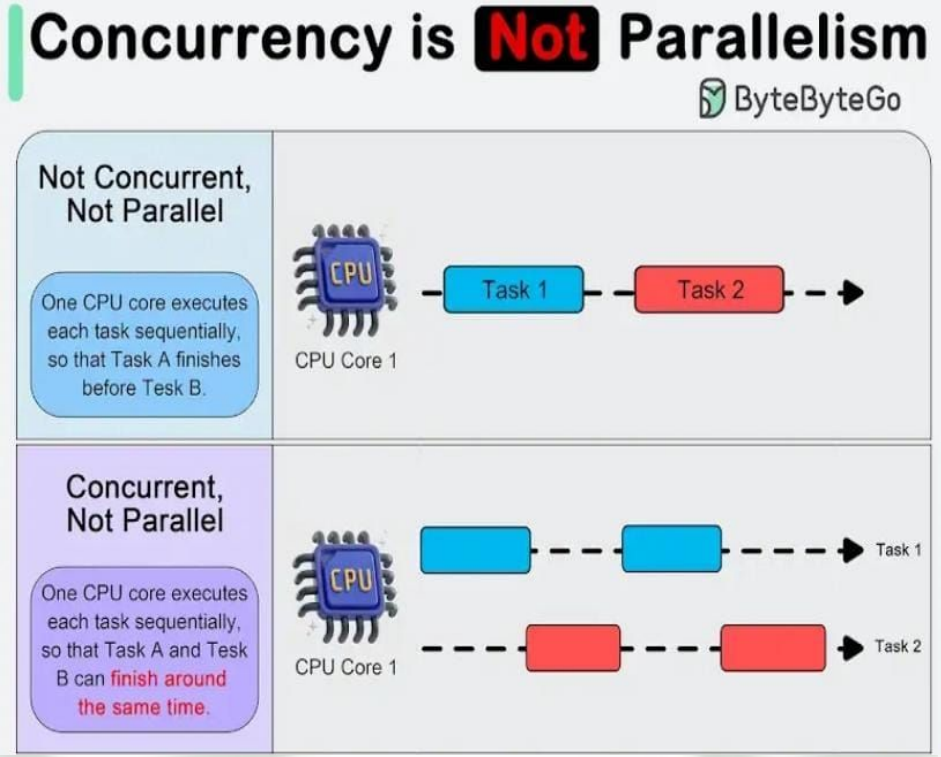
Sendo mais formal...

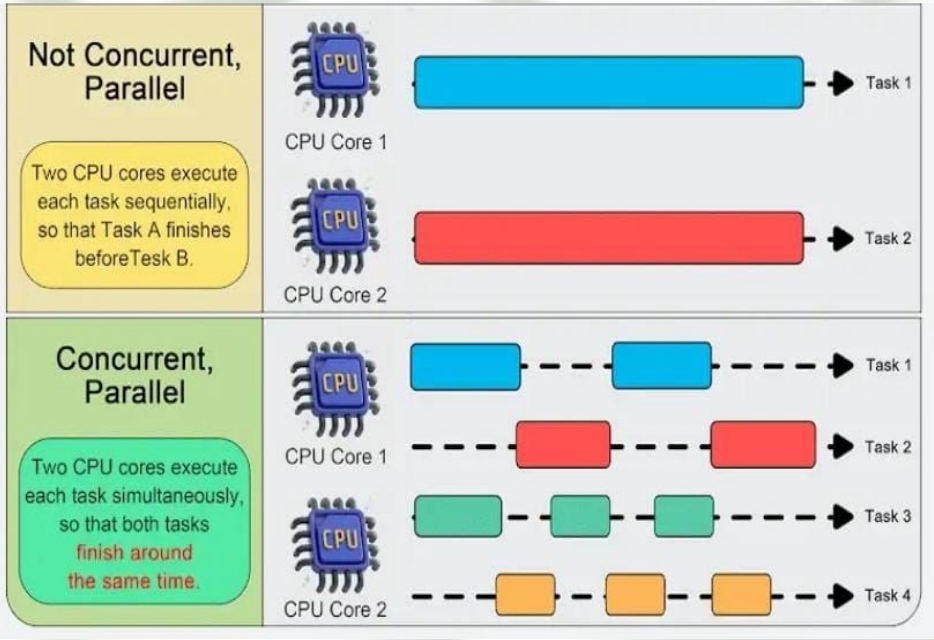
- ***Concorrência*** *é a capacidade de se executar duas ou mais tarefas em* ***um período de tempo****;*
- ***Paralelismo*** *é a capacidade de se executar duas ou mais tarefas de* ***ao mesmo tempo****;*

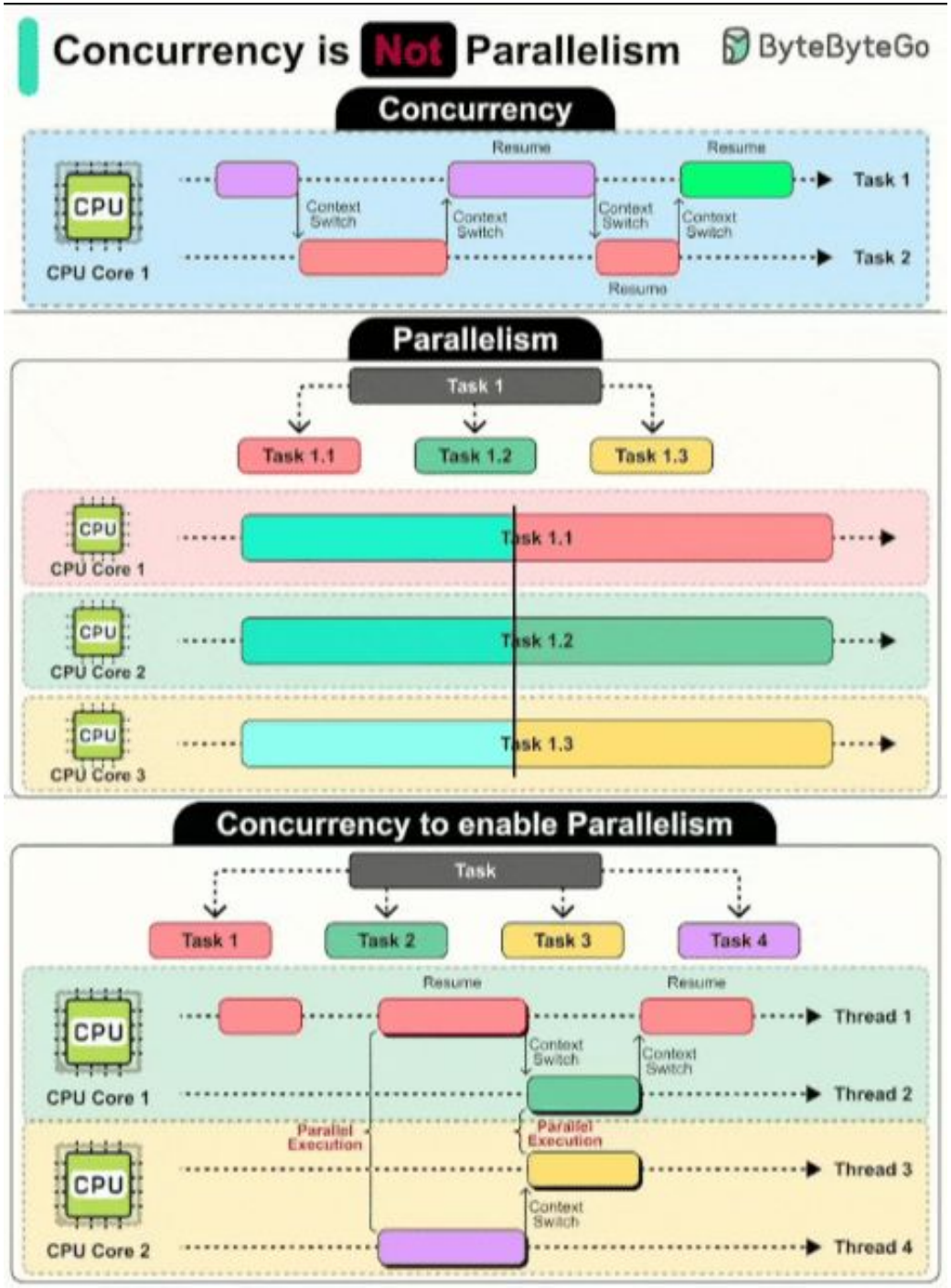


INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Sendo mais formal...



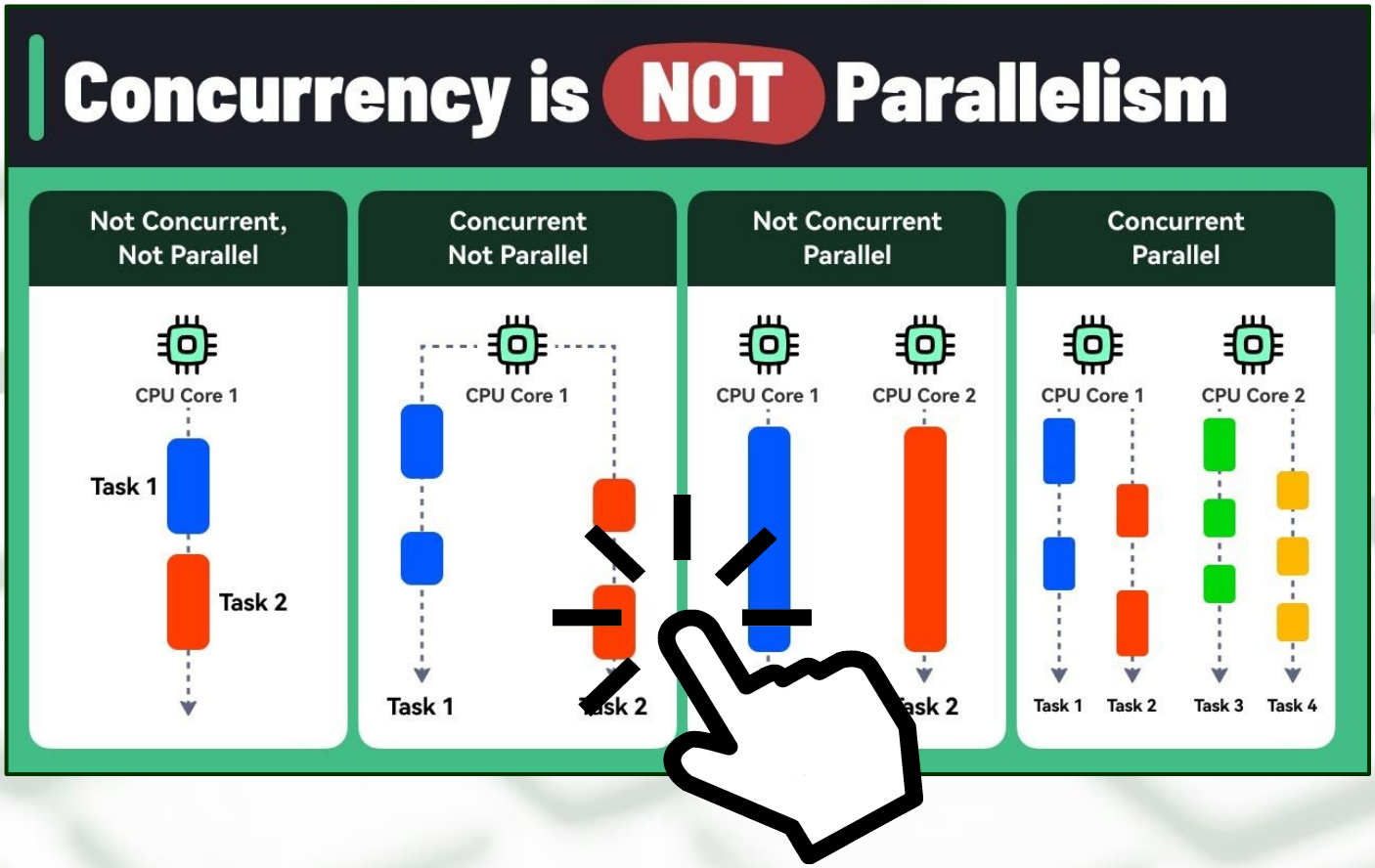



Concurrency is Not Parallelism
ByteByteGo
Concurrency
Resume
Resume
Task 1
Context Switch
Context Switch
Context Switch
Context Switch
CPU
CPU Core 1
Task 2
Resume
Parallelism
Task 1
Task 1.1
Task 1.2
Task 1.3
CPU
CPU Core 1
Task 1.1
CPU
CPU Core 2
Task 1.2
CPU
CPU Core 3
Task 1.3
Concurrency to enable Parallelism
Task
Task 1
Task 2
Task 3
Task 4
Resume
Resume
Thread 1
Context Switch
Context Switch
CPU
CPU Core 1
Thread 2
Parallel Execution
Parallel Execution
Thread 3
CPU
CPU Core 2
Context Switch
Thread 4

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Sendo mais formal...

- ***Assista mais detalhes em...***



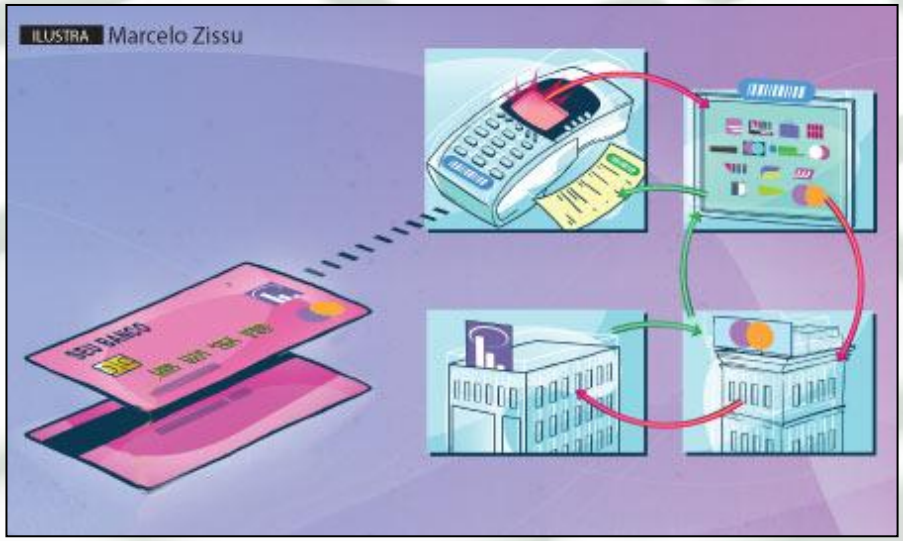
INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Computação Distribuída

- Um **aglomerado de computadores (ou sistemas)** trabalhando **em conjunto** e que se apresentam ao usuário **como um sistema único**.

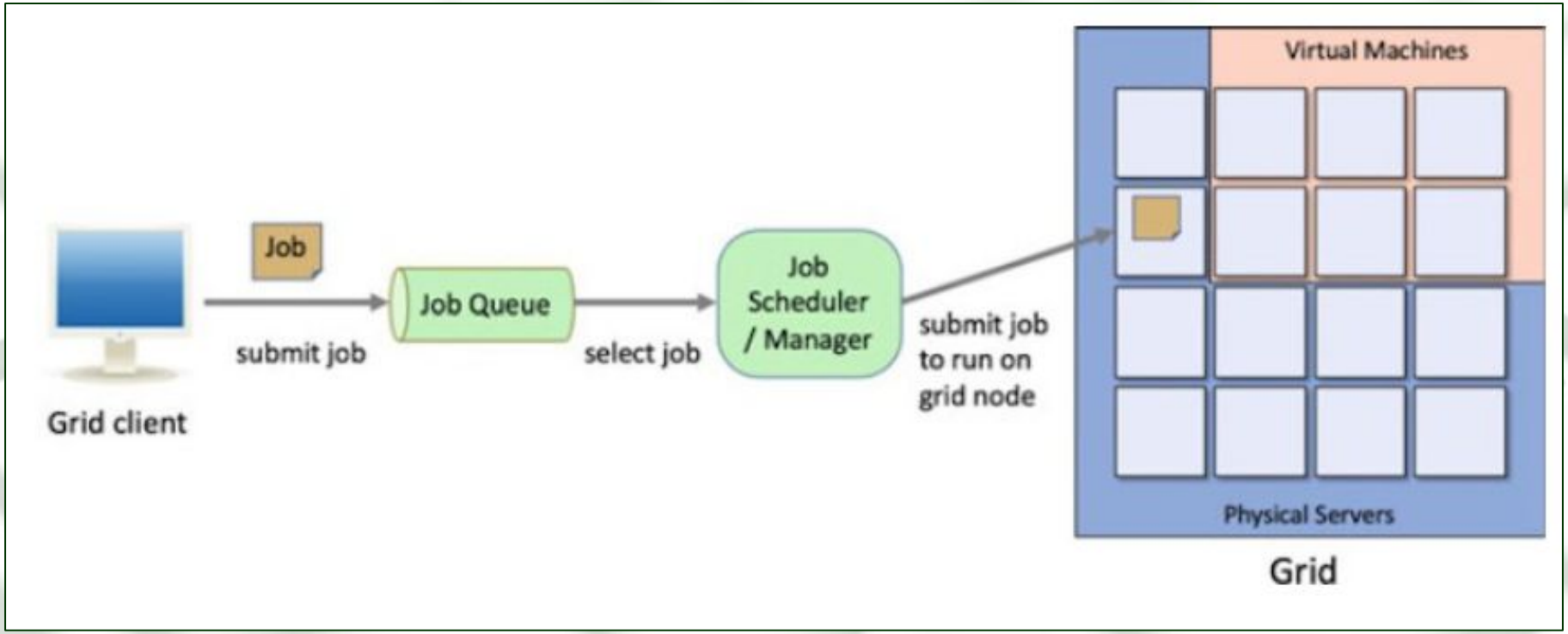
"Concorrência e Paralelismo para além da sua própria máquina".





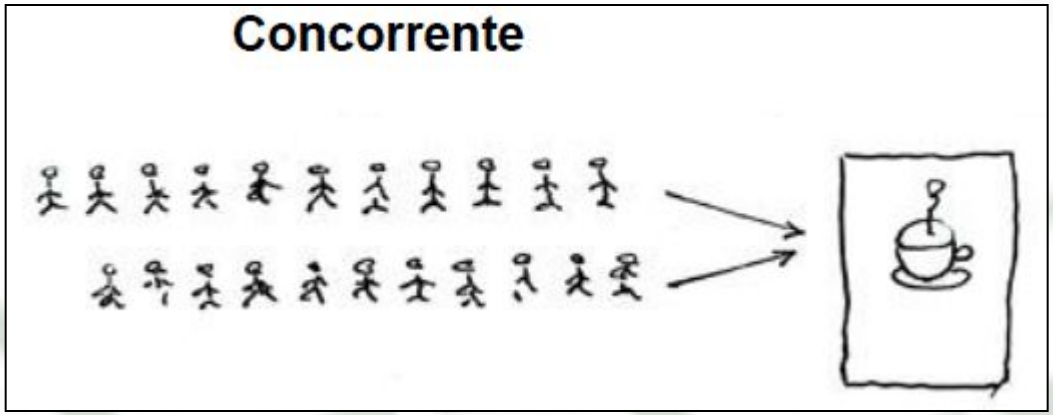
INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

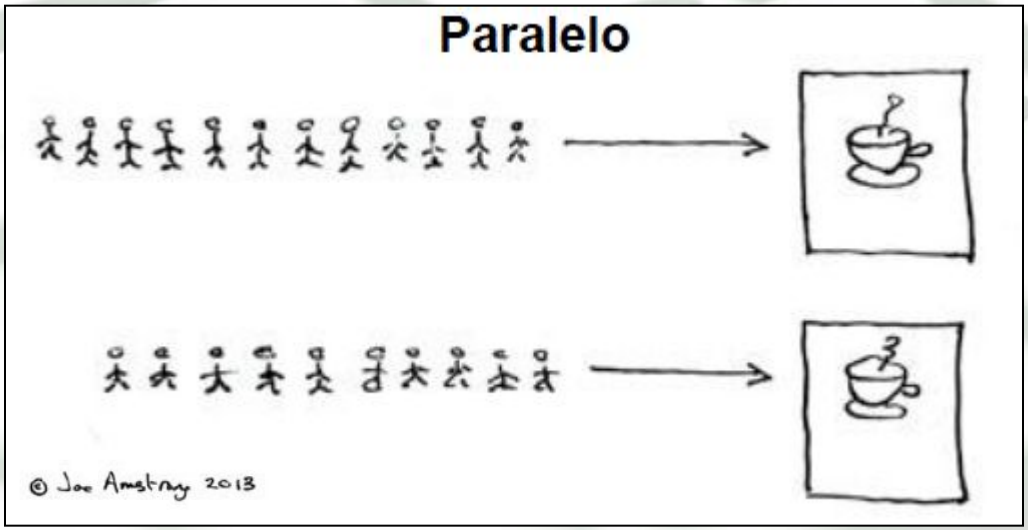
Computação Distribuída



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

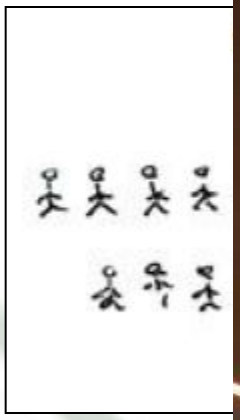
Resumindo...



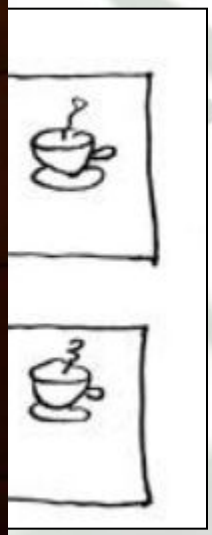


INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Resumindo...







Analisando um Caso…

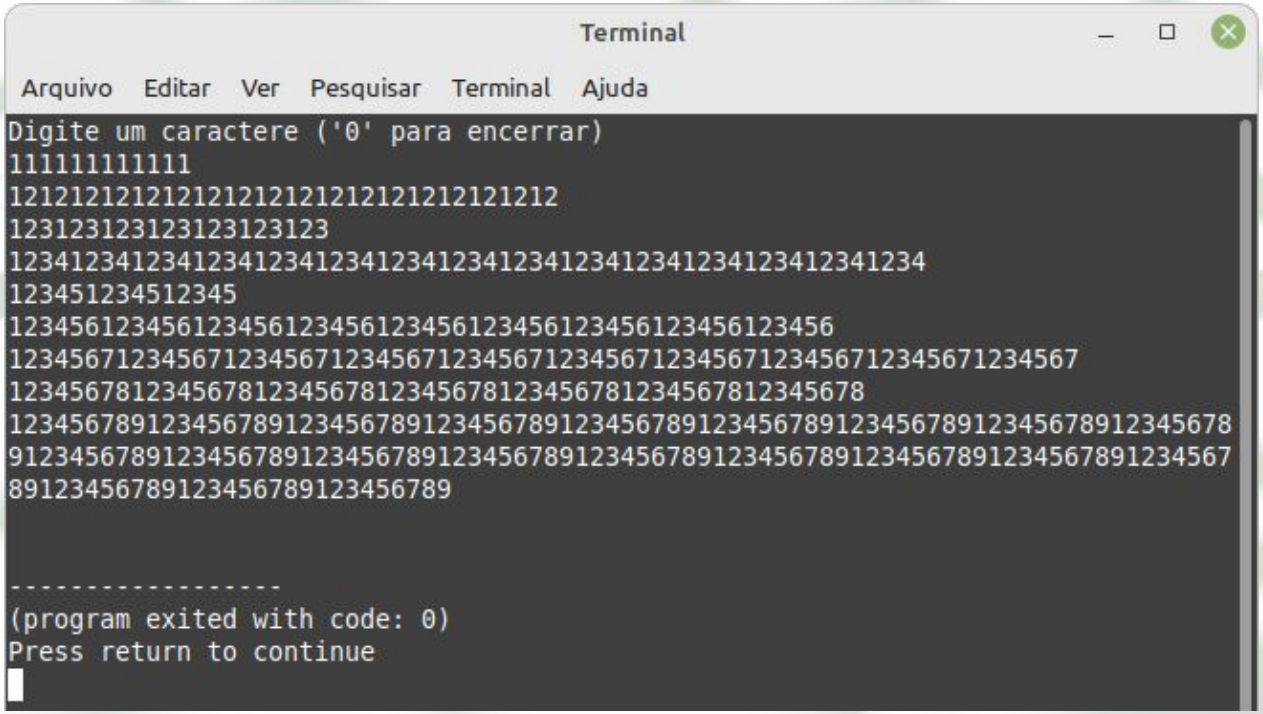
- Imagine o desenvolvimento de um servidor WEB, que deve oferecer concorrência para atender vários clientes simultaneamente…

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Analisando um Caso…

- **Imagine o desenvolvimento de um servidor WEB, que deve oferecer concorrência para atender vários clientes simultaneamente…**

Lembram-se?



Analisando um Caso…

- Imagine o desenvolvimento de um servidor WEB, que deve oferecer concorrência para atender vários clientes simultaneamente…

Seria interessante criar um novo processo para atender cada cliente que requisita a página?

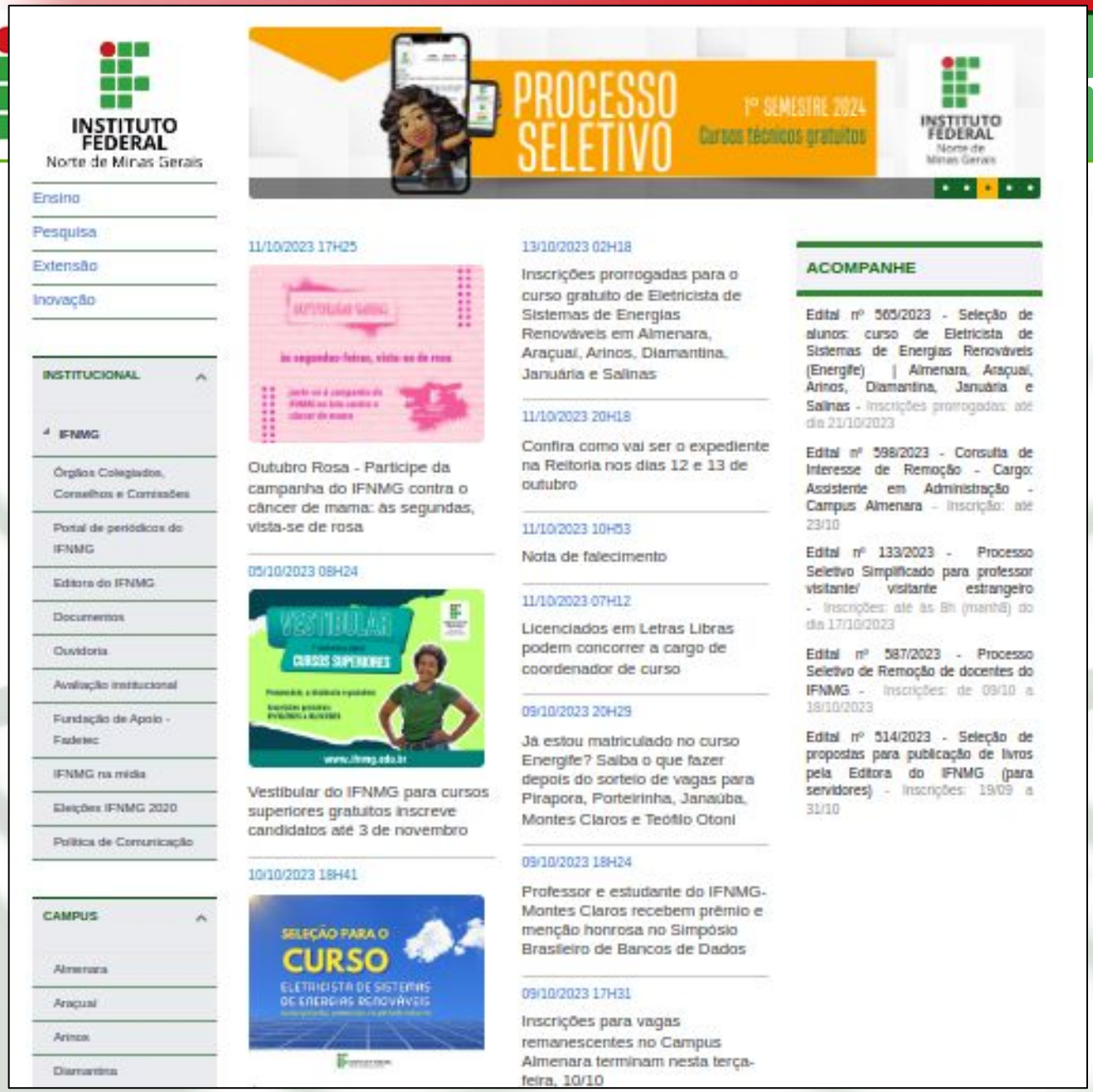
Analisando um Caso…

- **Imagine o desenvolvimento de um servidor WEB, que deve oferecer concorrência para atender vários clientes simultaneamente…**

Seria interessante criar um novo processo para atender cada cliente que requisita a página?

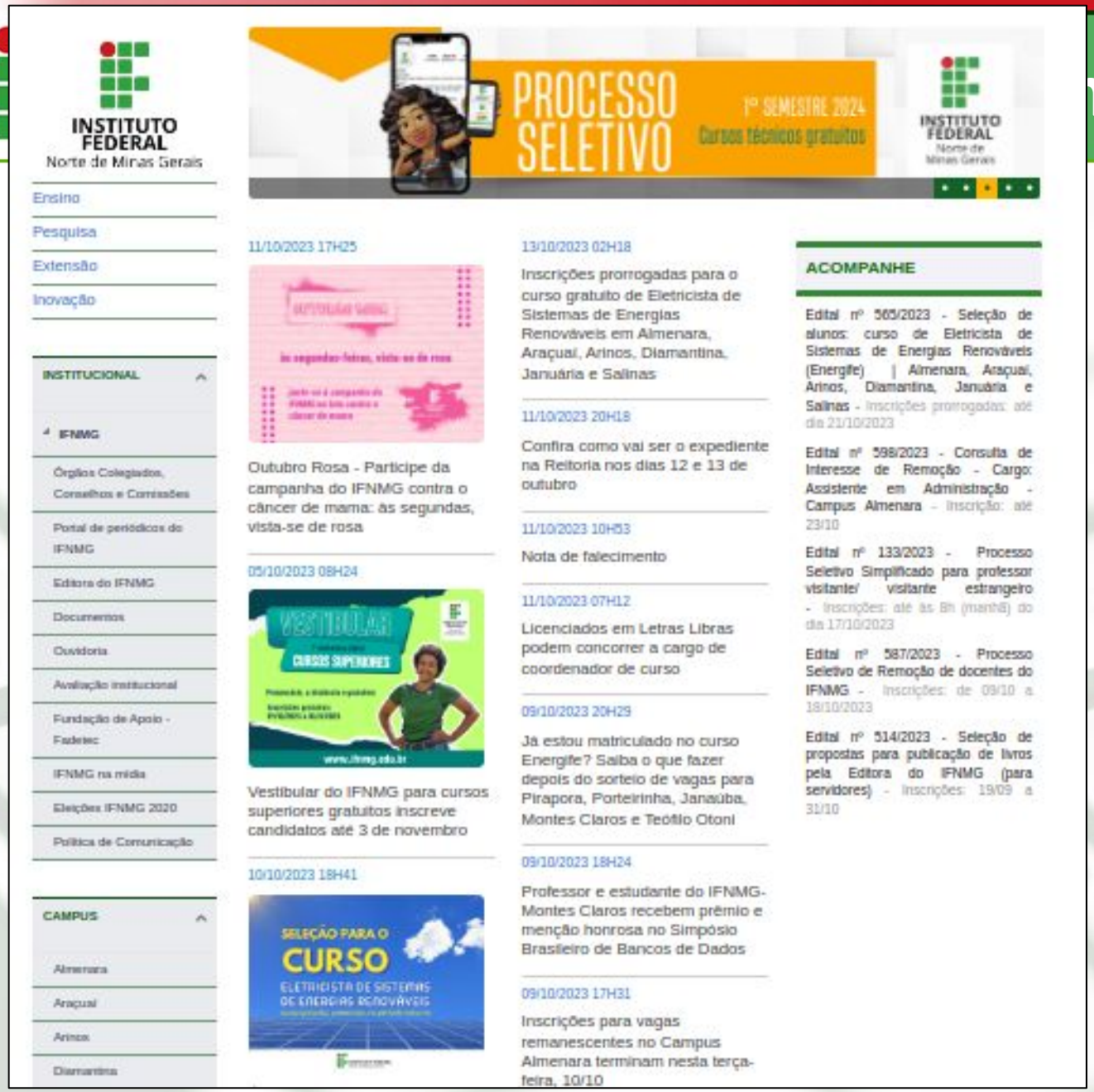
- Custo de memória: PCBs distintos e isolados.
- Custo para gerência dos processos:
  - Complexas trocas de contexto.
- No fim das contas, **a mesma página que será enviada a todos os clientes…**

m Caso…



Por outro lado… Pensemos no ***client-side…***

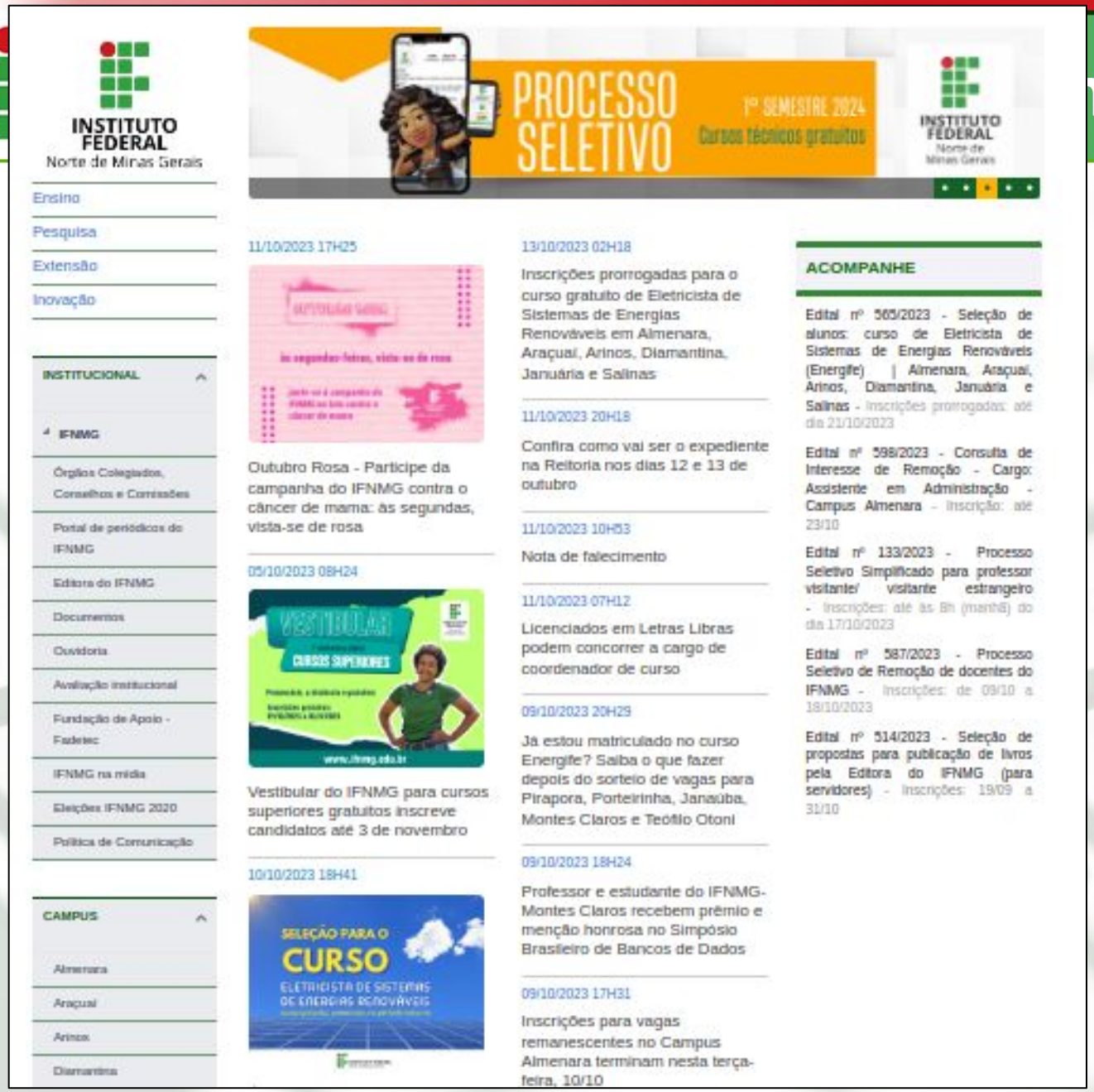
m Caso…



Por outro

Imagine se para toda página visitada, apenas uma imagem pudesse ser carregada e renderizada por vez…

m Caso...



Por outro

Imagine se para

Com concorrência, podemos fazer com que cada arquivo da página seja baixado e renderizado "ao mesmo tempo".

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Concorrência

- Mecanismos para implementação de **Concorrência**

Corrotinas

Libs de alto nível fornecidas pela linguagem de programação

Threads

Processos

Mecanismos nativos do Sistema Operacional

(Independe de linguagem)

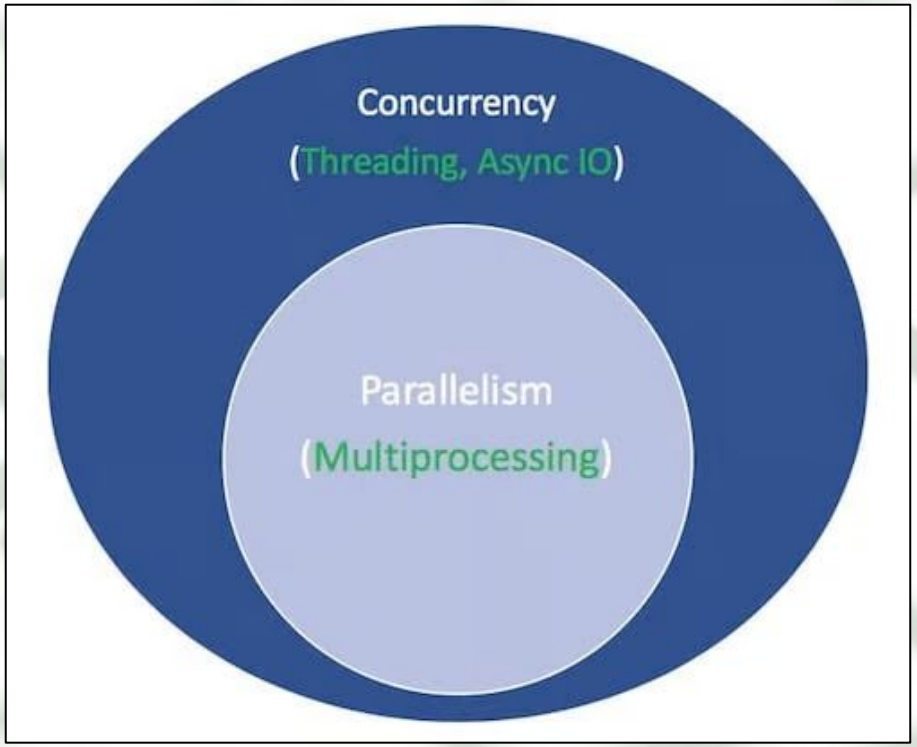
Concorrência

- Em Python…

Corrotinas
asyncio, trio, curio, …

Threads
threading

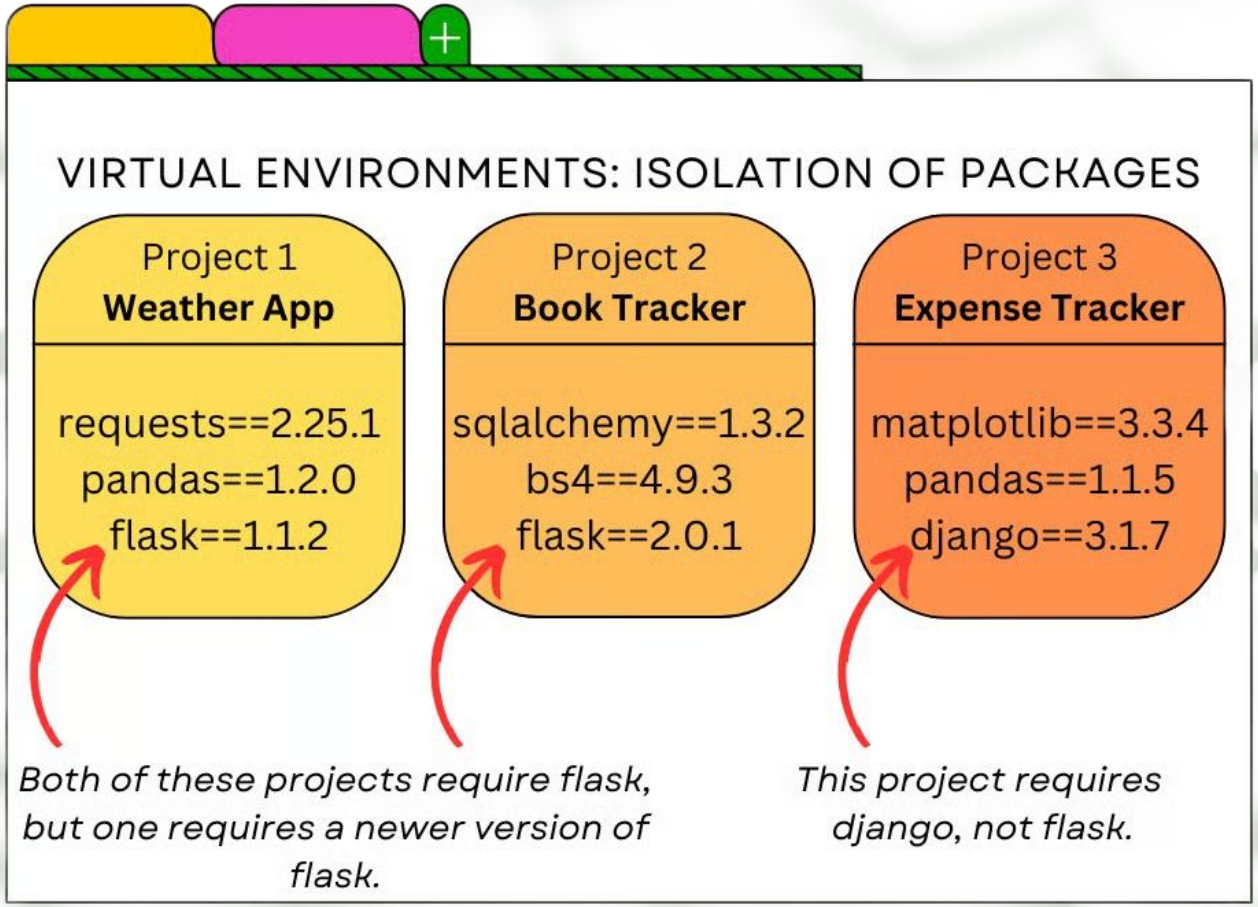
Processos
multiprocessing



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

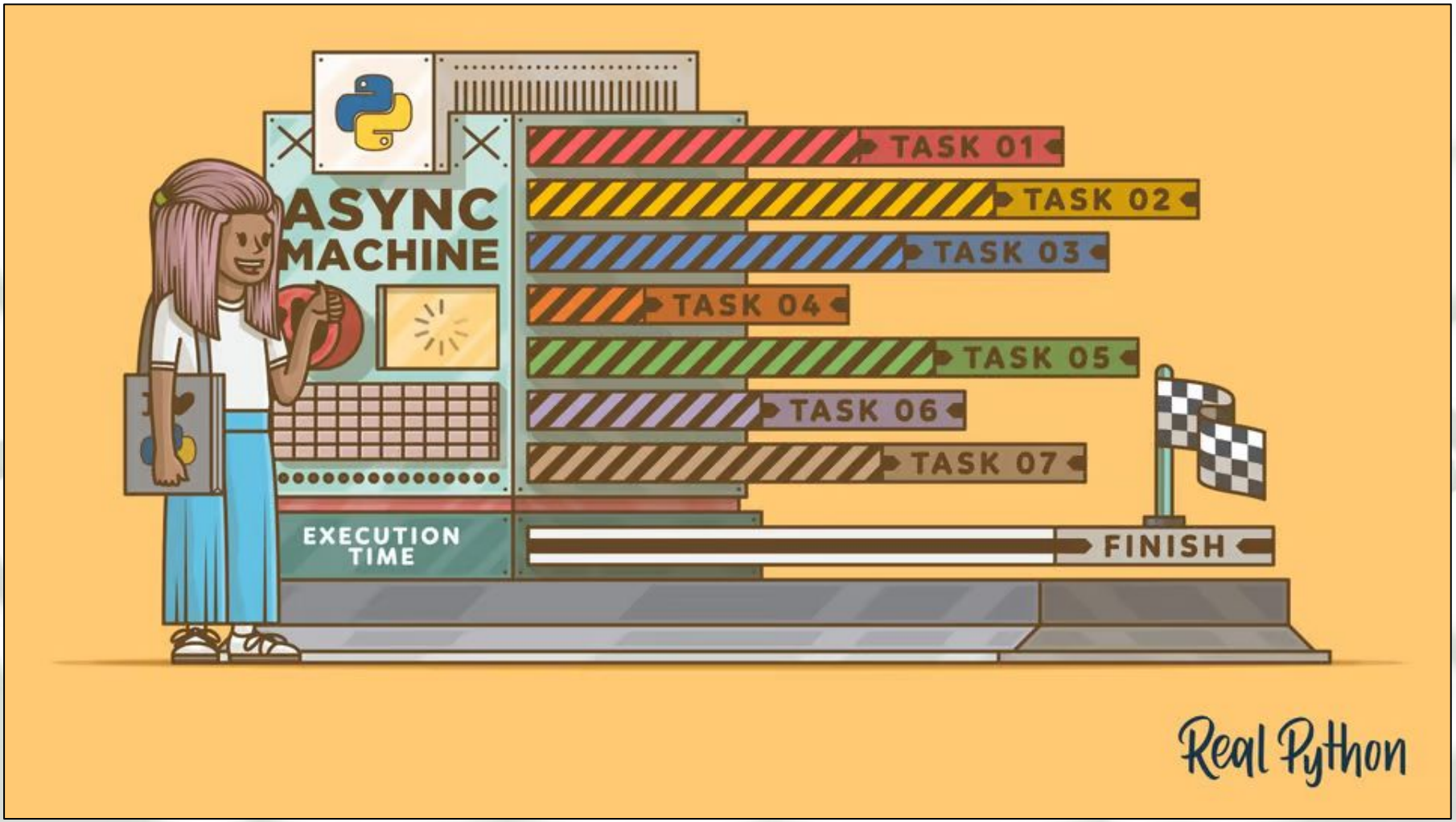
Ambientes Virtuais

- A Importância de Ambientes Virtuais



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Bora concorrer!



Funções Geradoras

- O fundamento básico por trás dos *frameworks* de corrotinas são as **Funções Geradoras**.
- **Função Geradora** é um tipo especial de função que consegue **manter o estado da sua execução**, mesmo após "retornar" um valor.

Funções Geradoras

- O fundamento básico por trás dos *frameworks* de corrotinas são as **Funções Geradoras**.
- **Função Geradora** é um tipo especial de função que consegue **manter o estado da sua execução**, mesmo após "~~retornar~~" um valor.

Embora **ceder** (***yield***) **um valor** estaria tecnicamente mais correto no âmbito das funções geradoras…

Vamos Entender Porque…

Leia mais AQUI

Funções Geradoras

- Imagine uma função que deve processar uma lista de 1.000 itens de um banco de dados…
  - **Abordagem Tradicional** => Levar todo o conteúdo (1.000 itens) para uma Estrutura de Dados em memória RAM (seja Array, Listas encadeadas…)
  - **Função Geradora** => “Gerar e Ceder” um item por vez e processá-lo individualmente.

Funções Geradoras

```
dataset = ['a','b','c','d','e','f','g','h',...,'xyz']


def geradora(ds):
 for item in ds:
     yield item


g = geradora(dataset)
for _ in range(4):
   print(str.capitalize(next(g)))
```

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Funções Geradoras

```
dataset = ['a','b','c','d','e','f','g','h',...,'xyz']


def geradora(ds):
 for item in ds:
     yield item

g = geradora(dataset)
for _ in range(4):
   print(str.capitalize(next(g)))
```

Onde a mágica acontece:

yield = ceder, passar a vez.

Funções Geradoras

```
dataset = ['a','b','c','d','e','f','g','h',...,'xyz']


def geradora(ds):
 for item in ds:
     yield item


g = geradora(dataset)
for _ in range(4):
   print(str.capitalize(next(g)))
```

Objetos processados individualmente, à medida em que são gerados.

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Funções Geradoras

```
from random import randint

def tradicional(inicio,fim):
    lst = []
    for _ in range(1_000):
        lst.append(randint(inicio,fim))
    return lst

def geradora(inicio,fim):
 for _ in range(1_000):
     yield randint(inicio,fim)
```

Abordagem Tradicional

vs.

Função Geradora

Retorna uma Lista inteira

Retorna um valor por vez

Funções Geradoras

```python
# FUNÇÃO TRADICIONAL
def potencia(n):
    result = []
    for i in range(n):
        result.append(2**i)
    return result

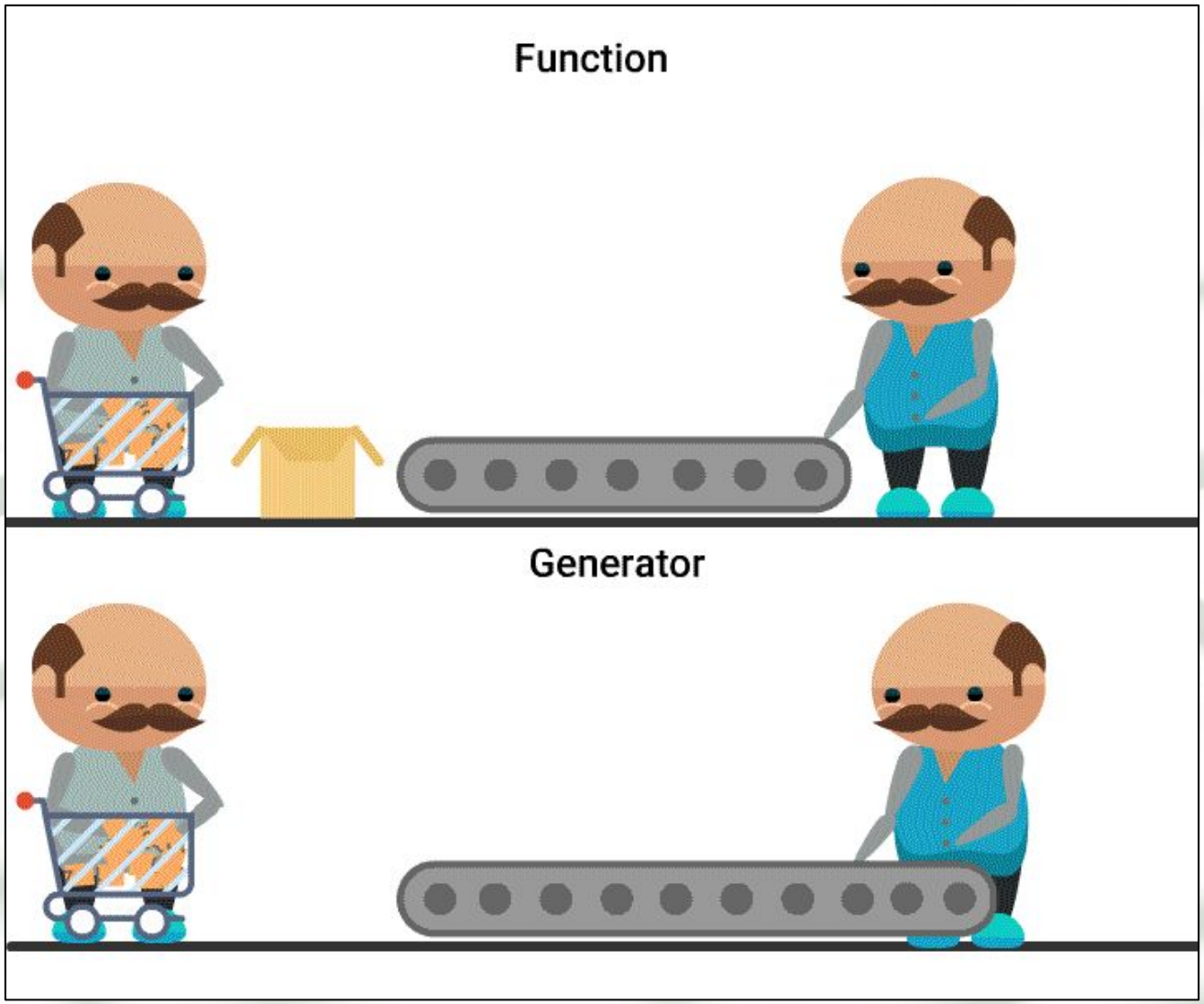
# FUNÇÃO GERADORA
def gerador_potencia(n):
    for i in range(n):
        yield 2**i

print(potencia(5))
print(list(gerador_potencia(5)))
```

Otimização do
CONSUMO DE MEMÓRIA

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Funções Geradoras



Funções Geradoras

Mas além dessa grande sacada,

as **Funções Geradoras** foram a base

de criação de algo **bem mais interessante...**

Funções Geradoras

Mas além dessa grande sacada,

as **Funções Geradoras** foram a base

de criação de algo **bem mais interessante...**

CORROTINAS ASSÍNCRONAS

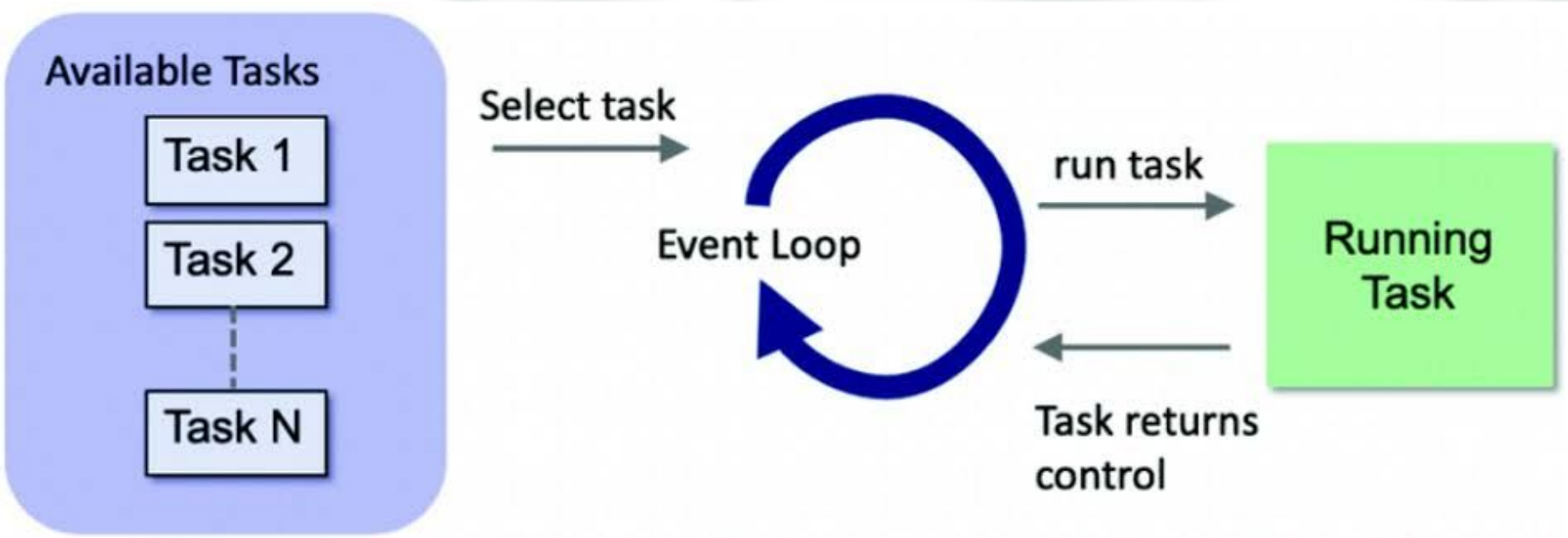
INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Corrotinas

- **Corrotina (*light-weight threads*)** é uma técnica especial (e relativamente nova) de se obter concorrência através de **tarefas assíncronas**.
- Ponto-chave do conceito é o **Loop de Eventos.**
- Imagine o **Loop de Eventos** como sendo um laço de iteração infinito, em que tarefas podem ser anexadas a este *loop*, e onde um ***framework*** possui autoridade para interromper e alternar a execução dessas tarefas, promovendo a concorrência.

Loop de Eventos

- **Loop de Eventos**



Corrotinas

- A técnica de **programação assíncrona por corrotinas** é inteiramente gerenciada pela aplicação (através de bibliotecas ou *frameworks* específicos).

O sistema operacional não detém qualquer influência (nem sequer conhecimento) sobre a concorrência realizada pelo Loop de Eventos.

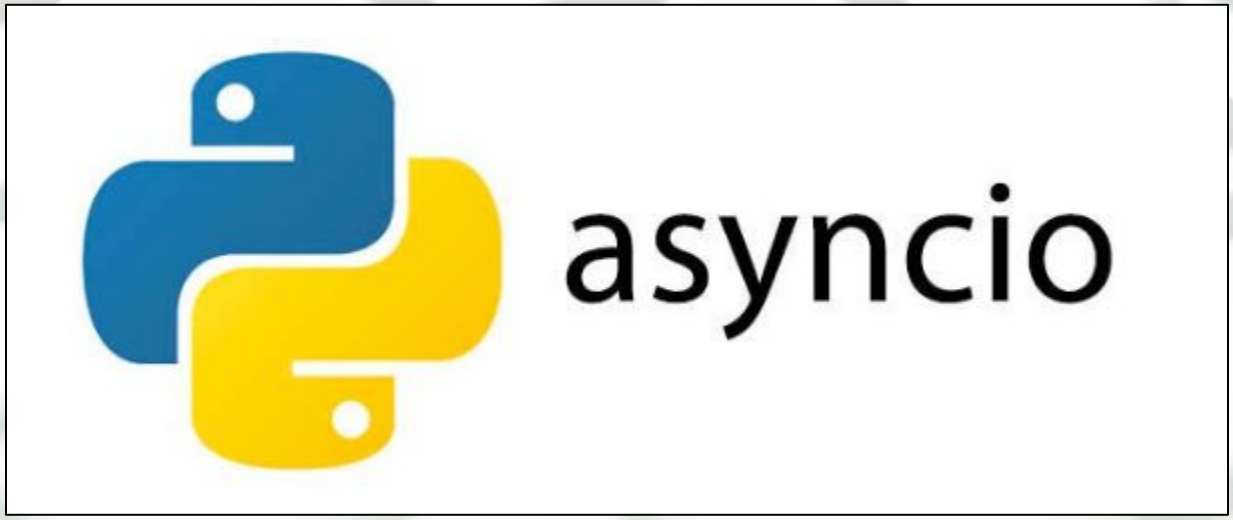
- Por este fato, a técnica de corrotina é, por padrão, executada em uma única thread/processo. **Sendo assim, a técnica torna-se inútil para problemas em que a solução seja a paralelização da solução.**

Corrotinas

- **Python**
  - asyncio
  - trio
  - curio
- **Java**
  - Quasar
- **Go / Golang**
  - Goroutines
- **Kotlin**
  - Coroutines

Asyncio

- **Asyncio (*asynchronous IO*)** é a principal biblioteca de corrotinas assíncronas do Python. Tornou-se nativa na linguagem desde a versão 3.7.
- Palavras chaves se tornaram reservadas: **async await**



Programa Síncrono

```
def task1():
  print("Task1 Iniciada")
  time.sleep(6)
  print("Task1 Concluída")

def task2():
  print("Task2 Iniciada")
  time.sleep(4)
  print("Task2 Concluída")

print('Aplicação Iniciada')
inicio = time.time()
task1()
task2()
print(f'Execução Finalizada em {time.time()-inicio:.2f} segs.')
```

```
#Executa algo que não dependa de você...
#p.ex: Replicação de DB
```

```
#Executa algo que não dependa de você...
#p.ex: Requisição API Rest
```



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Programa Síncrono

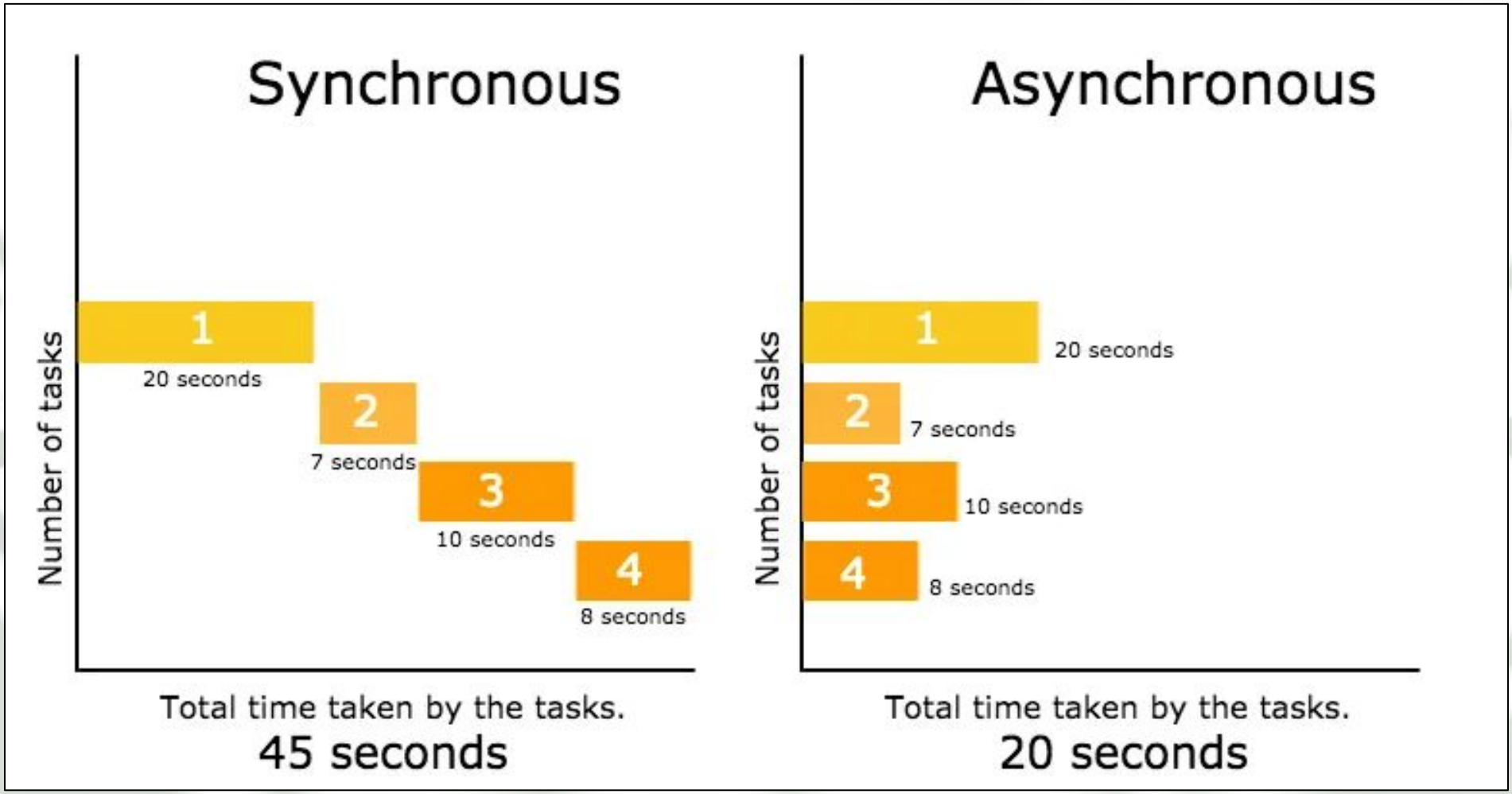
```
def task1():
  print("Task1 Iniciada")
  time.sleep(6)
```

```
Aplicação Iniciada
Task1 Iniciada
Task1 Concluída
Task2 Iniciada
Task2 Concluída
Execução Finalizada em 10.00 segs.
```

```
task1()
task2()
print(f'Execução Finalizada em {time.time()-inicio:.2f} segs.')
```

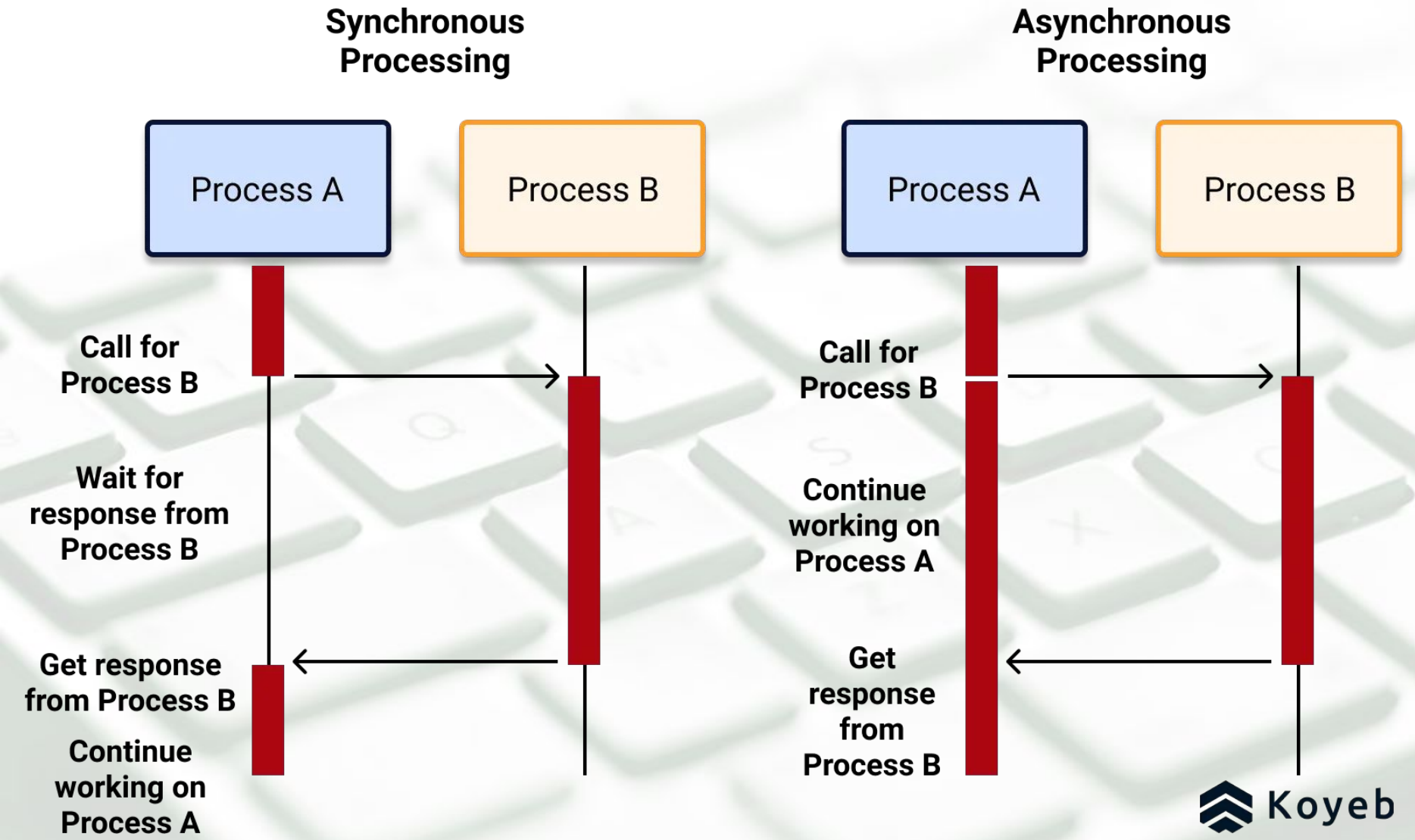
INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Asyncio



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Asyncio



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Asyncio

```
import asyncio

async def task1():
  print("Task1 Iniciada")
  time.sleep(6)
  print("Task1 Concluída")

async def task2():
  print("Task2 Iniciada")
  time.sleep(4)
  print("Task2 Concluída")

async def main():
  print('Aplicação Iniciada')
  inicio = time.time()
  task1()
  task2()
  print(f'Execução Finalizada em {time.time()-inicio:.2f} segs.')

asyncio.run(main())
```

Todas funções concorrentes devem ser registradas como assíncronas, através da *keyword* **async**

asyncio.run() cria um loop de eventos para a corrotina `main()`



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Asyncio

```
import asyncio

async def task1():
  print("Task1 Iniciada")
  time.sleep(6)
  print("Task1 Concluída")

async def task2():
  print("Task2 Iniciada")
  time.sleep(4)
  print("Task2 Concluída")

async def main():
  print('Aplicação Iniciada')
  inicio = time.time()
  task1()
  task2()
  print(f'Execução Finalizada em {time.time()-inicio:.2f} segs.')

asyncio.run(main())
```

Assim como nas funções geradoras, uma vez que uma função é definida como assíncrona (***async***), elas **NÃO** podem mais ser invocadas como qualquer outra…

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Asyncio

- Qualquer função assíncrona (corrotina) só é executada **quando agendada em algum Loop de Eventos**.



Asyncio

- Basicamente, existem 2 formas de agendar a execução de funções assíncronas no Loop de Eventos:

`await funcao()`

- Aguardando pela execução da corrotina `funcao()`
- Esse método é **bloqueante (síncrono)**.

`asyncio.create_task(funcao())`

- Criando uma tarefa para executar a corrotina `funcao()`
- Esse método é **concorrente (assíncrono)**.

Asyncio

```
import asyncio

async def task1():
  print("Task1 Iniciada")
  time.sleep(6)
  print("Task1 Concluída")

async def task2():
  print("Task2 Iniciada")
  time.sleep(4)
  print("Task2 Concluída")

async def main():
  print('Aplicação Iniciada')
  inicio = time.time()
  asyncio.create_task(task1())
  asyncio.create_task(task2())
  print(f'Execução Finalizada em {time.time()-inicio:.2f} segs.')

asyncio.run(main())
```

Neste exemplo, como queremos executar **task1()** e **task2()** concorrentemente, devemos criar tarefas associadas a essas funções. Quando uma task é criada, a inicialização é imediata.

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Asyncio

```python
import asyncio

async def task1():
  print("Task1 Iniciada")
  time.sleep(6)
  print("Task1 Concluída")

async def task2():
  print("Task2 Iniciada")
  time.sleep(4)
  print("Task2 Concluída")

async def main():
  print('Aplicação Iniciada')
  inicio = time.time()
  t1 = asyncio.create_task(task1())
  t2 = asyncio.create_task(task2())
  await t1
  await t2
  print(f'Execução Finalizada em {time.time()-inicio:.2f} segs.')

asyncio.run(main())
```

main() também é uma função assíncrona, e portanto, também está concorrendo com as tasks criadas.
É interessante aguardar a finalização das tasks lançadas por ela…
A keyword **await** é um mecanismo de sincronização de tarefas.
await bloqueia a execução de uma task até a finalização de outra(s).

Asyncio

```
Aplicação Iniciada
Task1 Iniciada
Task1 Concluída
Task2 Iniciada
Task2 Concluída
Execução Finalizada em 10.00 segs.
```

mas algo de errado
não está certo!!!

Asyncio

- Como visto antes, a técnica de **corrotina não implementa uma concorrência tradicional** - criada e supervisionada pelo Sistema Operacional - mas sim, um mecanismo gerenciado pela própria linguagem através de libs específicas.
- Isso produz um **dano colateral**, que é quando precisamos utilizar algum **recurso (lib, função ou método) que não suporta a técnica de corrotinas.**
- Nesta situação, **o efeito concorrente das tarefas** é anulado devido **este método/função não ser assíncrono também**.

Asyncio

- No exemplo anterior, a função

  `time.sleep()`

  não é assíncrona (não foi implementado como **async**) e portanto, **continua sendo um método tradicional (bloqueante).**

- Mesmo que a função que a invoca seja assíncrona, não tira o fato que o `time.sleep()` **seja uma função bloqueante do Loop de Eventos.**

Asyncio

- A solução para estes casos é buscar alternativas de funções que executam a mesma rotina, mas que também ofereçam suporte a **programação assíncrona**.

- A própria lib asyncio contém várias funções adaptadas ao uso assíncrono, como:
  - **`asyncio.sleep()`**

- Já outras libs foram aprimoradas para suportar programação assíncrona, como: **aioconsole**, **httpx**, **...**

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Asyncio

- A solução para estes casos é buscar alternativas de

O problema com as abordagens usuais da programação assíncrona é que elas são propostas do tipo "TUDO OU NADA"....
Ou você reescreve todo o código, de forma que nada nele bloqueie [o processamento] ou você estará só perdendo tempo.

VAN STEEN, Maarten; TANENBAUM, Andrew S. Distributed Systems. 4th Edition.

-
-
- programação assíncrona, como: **aioconsole, httpx, ...**

INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Asyncio

```
import asyncio

async def task1():
  print("Task1 Iniciada")
  await asyncio.sleep(6)
  print("Task1 Concluída")

async def task2():
  print("Task2 Iniciada")
  await asyncio.sleep(6)
  print("Task2 Concluída")

async def main():
  print('Aplicação Iniciada')
  inicio = time.time()
  t1 = asyncio.create_task(task1())
  t2 = asyncio.create_task(task2())
  await t1
  await t2
  print(f'Execução Finalizada em {time.time()-inicio:.2f} segs.')

asyncio.run(main())
```



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Asyncio

```
import asyncio

async def task1():
  print("Task1 Iniciada")
  await asyncio.sleep(6)
```

```
Aplicação Iniciada
Task1 Iniciada
Task2 Iniciada
Task2 Concluída
Task1 Concluída
Execução Finalizada em 6.00 segs.
```

Porque a Task 2 terminou antes da Task 1?

```
asyncio.run(main())
```

Pesquise sobre…

- É possível executar uma função assíncrona através da keyword **`await`**, sem que antes tenha sido criada uma `task`? Qual a consequência dessa ação?
- Uma função **async** pode receber valores via parâmetros? Como invocar uma task assíncrona enviando dados?
- Uma função **async** pode retornar valor como resultado? Como obter esse valor a partir da sua invocação?
- É possível que duas ou mais tasks assíncronas compartilhem uma mesma estrutura de dados para leitura e escrita (p.ex. Lista)?

Pesquise sobre...

- Pesquise sobre os seguintes métodos da lib **`asyncio`**. Estude as diferenças e aplicabilidades de cada método.

```
asyncio.gather()

async with asyncio.TaskGroup() as tg

asyncio.wait()

asyncio.wait_for()
```

- Implemente um programa diferente que aplique cada um dos métodos acima.

Referências

- ***VAN STEEN, Maarten; TANENBAUM, Andrew S. Distributed Systems. 4th Edition.*** https://www.distributed-systems.net/index.php/books/ds4/

- ***Basics of Asyncio in Python***
  https://raccoon.ninja/pt/post/dev/basics-of-asyncio-in-python/

- ***Async IO in Python: A Complete Walkthrough***
  https://realpython.com/async-io-python/